Unit-8 (Classification, prediction, and Clustering)

BScCSIT 8th Semester

Lecturer : Mukesh Prasad Chaudhary

Classification vs. Prediction vs. Estimation

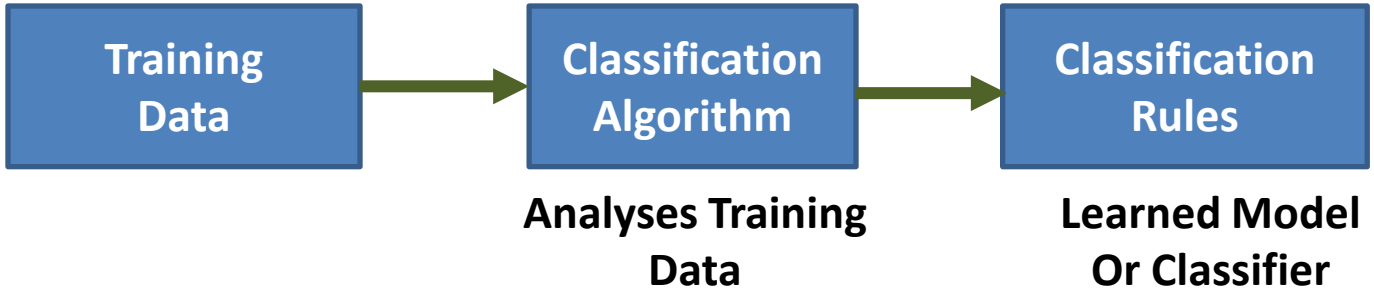
- **Classification**
  - predicts categorical class labels (discrete or nominal)
  - classifies data (constructs a model) based on the training set and the values (class labels) in a classifying attribute and uses it in classifying new data
  - E.g., grouping patients based on their medical history.
  - E.g., grouping students based on their test scores.
- **Prediction**
  - Prediction is the use of existing data values to guess a future value.
  - models continuous-valued functions, i.e., predicts unknown or missing values
  - E.g., using a patient's medical history to guess the effect of a treatment.
  - E.g., using a student's past test scores to guess chance of success in a future exam.
- **Estimation** is the prediction of a continuous a value.

– E.g., predicting a patient's cholesterol level given certain drugs.

– E.g., predicting a student's score on the GRE knowing her scores on school tests.

Classification

Classification can be described by a two step process given in appended block diagram:



Step 1

•Also known as "supervised learning" as class labels are known. It is different than "Unsupervised learning" or clustering where class labels are not known.

•A model is built describing a predetermined set of data classes or concepts.

•The model is constructed by analyzing database tuples described by their attributes.

•Each tuple is assumed to belong to a predefined class and called as a class label attribute.

•Data tuples are also referred as Samples, Examples or Objects.

•Data tuples selected randomly form a training data set and are called training samples.

• The learning of the model is termed as" Supervised " as it is told which class the training sample belongs.

•This is in contrast to Clustering which is termed unsupervised learning.

Step 2

Test data verifies the accuracy of Classification Rules

The model is used for classification. First the predictive accuracy of the model is estimated. If the accuracy is acceptable the model can be used to classify future tuples or objects for which class label is not known.

Classification

- It is the process of finding a model (or function) that describes and distinguishes data classes or concepts, for the purpose of being able to use the model to predict the class of objects whose class label is unknown.
- The derived model is based on the analysis of a set of training data (i.e., data objects whose class label is known).

"How is the derived model presented?" The derived model may be represented in various forms, such as *classification (IF-THEN) rules*, *decision trees*, *mathematical formulae*, or *neural networks*.

(a)

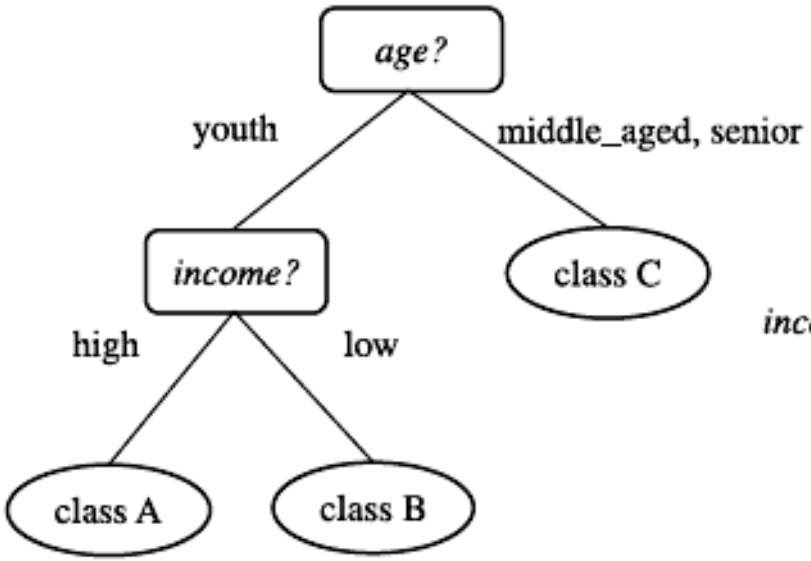
age(X, "youth") AND income(X, "high") → class(X, "A")

age(X, "youth") AND income(X, "low") → class(X, "B")

age(X, "middle_aged") → class(X, "C")

age(X, "senior") → class(X, "C")

(b)



(c)

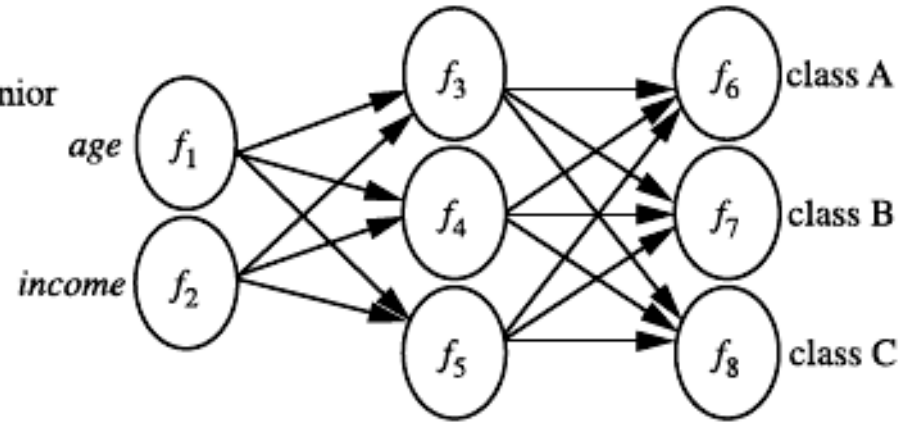


Figure : A classification model can be represented in various forms, such as (a) IF-THEN rules, (b) a decision tree, or a (c) neural network.

Classification : Example of Grading

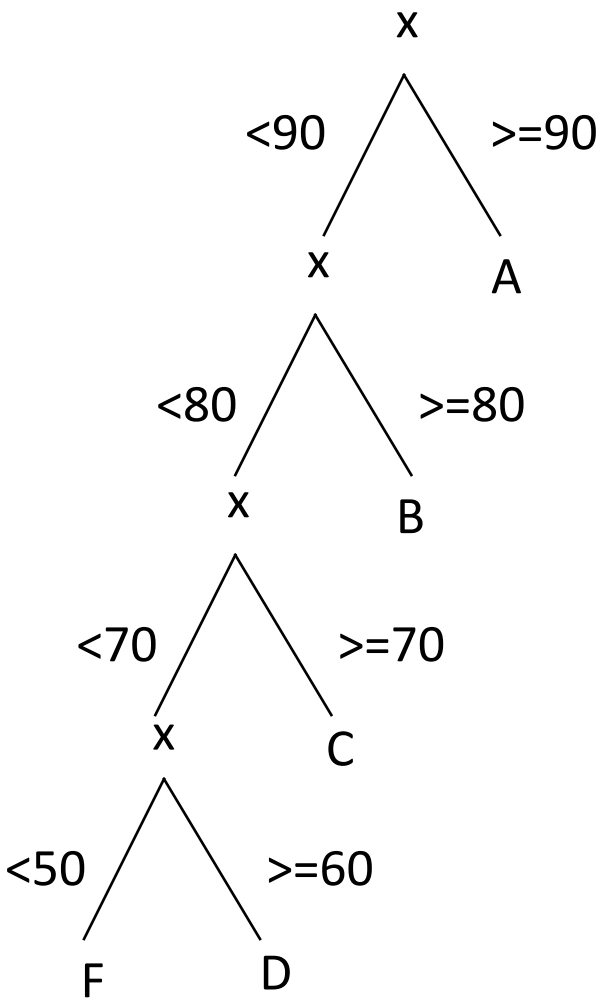
If x >= 90 then grade =A.

If 80<=x<90 then grade =B.

If 70<=x<80 then grade =C.

If 60<=x<70 then grade =D.

If x<50 then grade =F.



How does classification work?

Data classification is a two-step process:

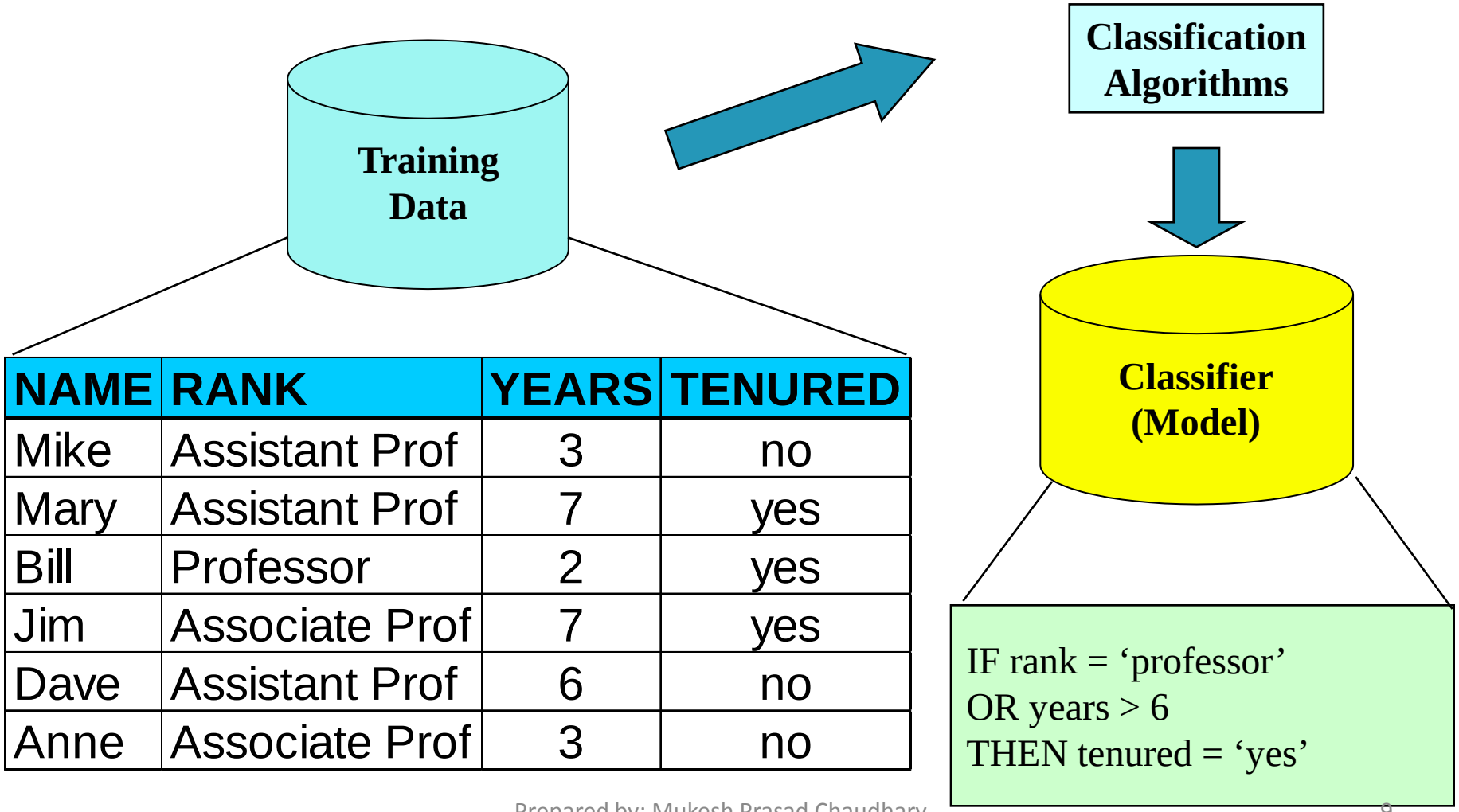
(1) Model construction

Training data are analyzed by a classification algorithm. A classifier is built describing a predetermined set of data classes or concepts. Also called as training phase or learning stage.

(2) Model usage

Test data are used to estimate the accuracy of the classification rules. If the accuracy is considered acceptable, the rules can be applied to the classification of new data tuples.

Model Construction



NAME	RANK	YEARS	TENURED
Mike	Assistant Prof	3	no
Mary	Assistant Prof	7	yes
Bill	Professor	2	yes
Jim	Associate Prof	7	yes
Dave	Assistant Prof	6	no
Anne	Associate Prof	3	no

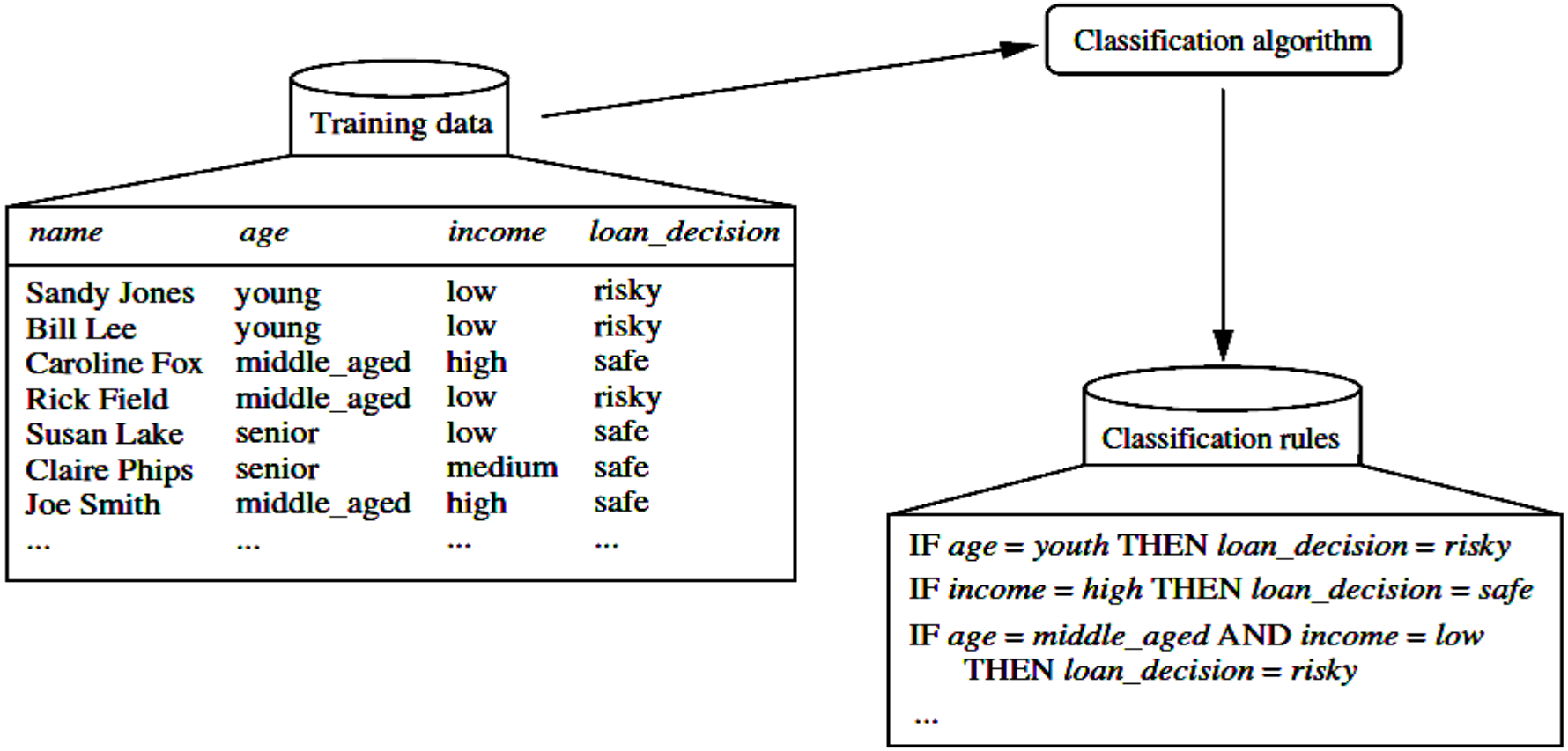
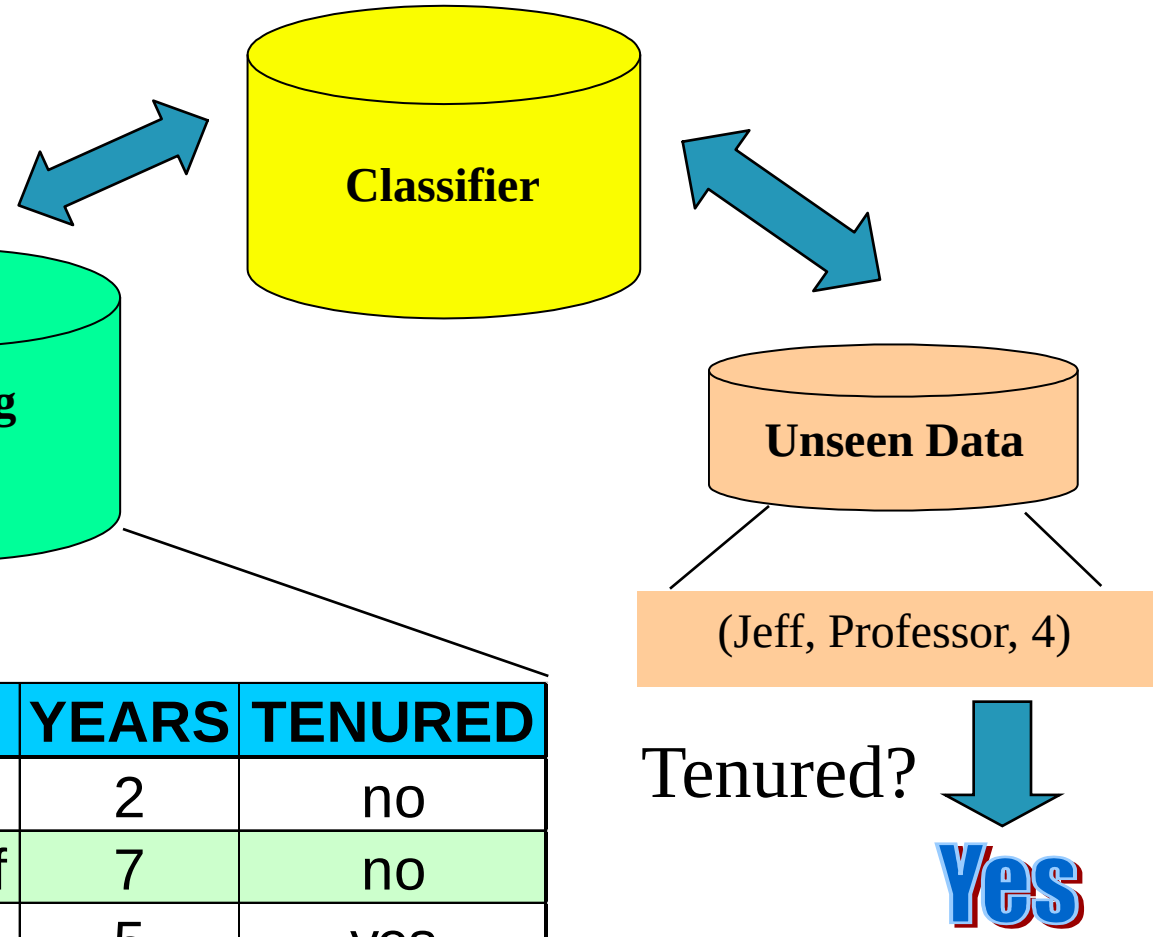


Figure: Learning

Here, the class label attribute is *loan decision*, and the learned model or classifier is represented in the form of classification rules.

Model Usage



NAME	RANK	YEARS	TENURED
Tom	Assistant Prof	2	no
Merlisa	Associate Prof	7	no
George	Professor	5	yes
Joseph	Assistant Prof	7	yes

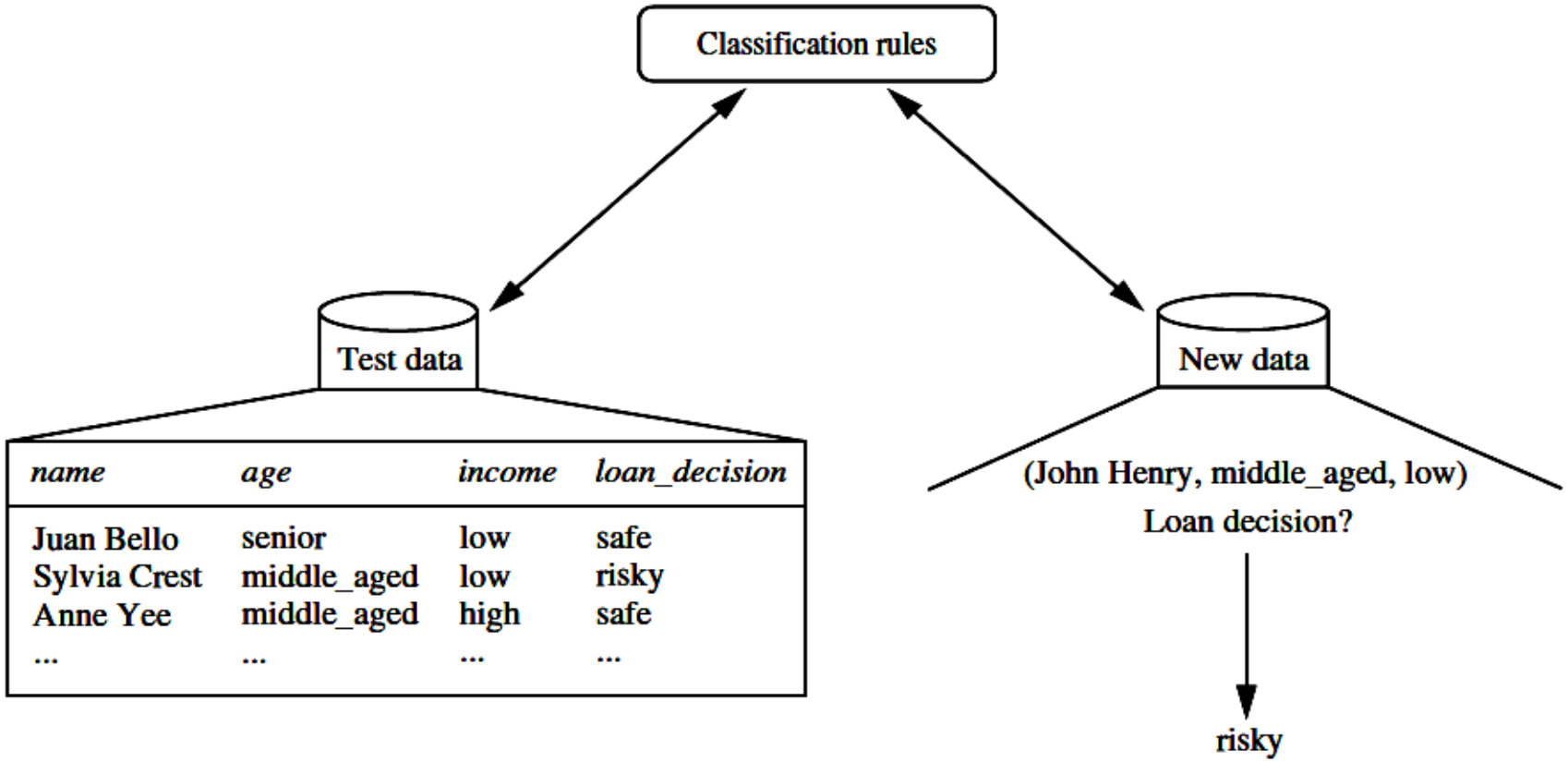


Figure: Classification

Supervised vs. Unsupervised Learning

- Supervised learning (classification)
  - Supervision: The training data (observations, measurements, etc.) are accompanied by labels indicating the class of the observations
  - New data is classified based on the training set
- Unsupervised learning (clustering)
  - The class labels of training data is unknown
  - Given a set of measurements, observations, etc. with the aim of establishing the existence of classes or clusters in the data

Classification and Prediction: Issues

- The major issue is **preparing** the data for Classification and Prediction are:
- **Data Cleaning**
- **Relevance Analysis**
- **Data Transformation and reduction**
  - ❖**Normalization**
  - ❖**Generalization**

Classification and Prediction: Issues

- **Data Cleaning**
  - ❖Data cleaning involves removing the noise and treatment of missing values.
  - ❖The noise is removed by applying smoothing techniques and the problem of missing values is solved by replacing a missing value with most commonly occurring value for that attribute.
- **Relevance Analysis**
  - ❖Database may also have the irrelevant attributes. Correlation analysis is used to know whether any two given attributes are related.

Classification and Prediction: Issues

- **Data Transformation and reduction**

  The data can be transformed by any of the following methods:

  Normalization - The data is transformed using normalization. Normalization involves scaling all values for given attribute in order to make them fall within a small specified range. Normalization is used when in the learning step, the neural networks or the methods involving measurements are used.

  Generalization -The data can also be transformed by generalizing it to the higher concept. For this purpose we can use the concept hierarchies.

- **Note:** Data can also be reduced by some other methods such as wavelet transformation, binning, histogram analysis, and clustering.

Issues(2): Evaluating Classification Methods

- **Accuracy**
  - classifier accuracy: predicting class label
  - predictor accuracy: guessing value of predicted attributes
- **Speed**
  - time to construct the model (training time)
  - time to use the model (classification/prediction time)
- **Robustness**: handling noise and missing values
- **Scalability:** efficiency in disk-resident databases
- **Interpretability**
  - understanding and insight provided by the model
- **Other measures**, e.g., goodness of rules, such as decision tree size or compactness of classification rules

Decision Trees

A decision tree is a predictive model that as its name implies can be viewed as a tree. Specifically each branch of the tree is a classification question and the leaves are partitions of data set with their classification.

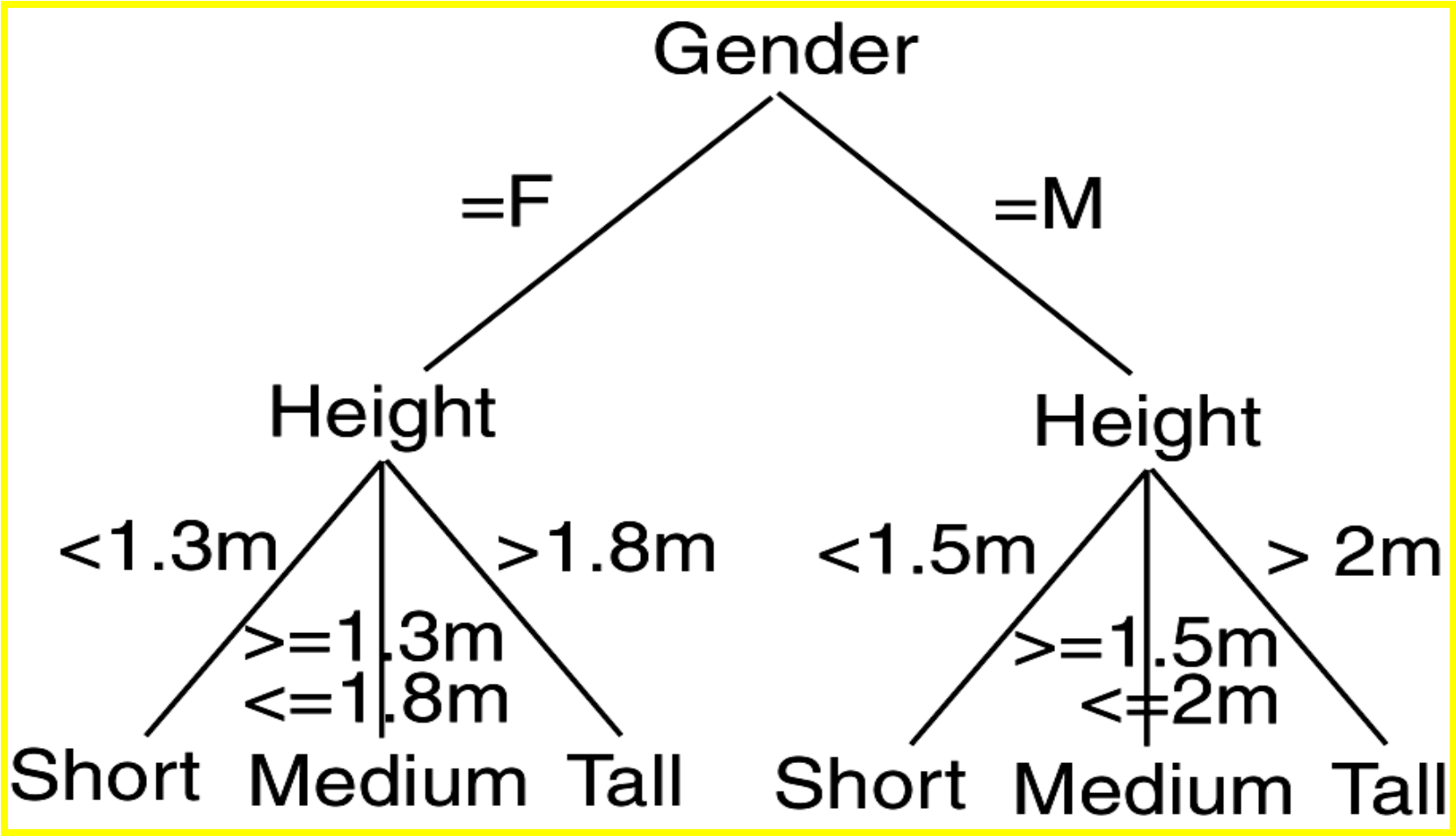
A decision tree makes a prediction on the basis of a series of decisions. The decision trees are being built on historical data and are a part of the supervised learning. The machine learning technique for inducting a decision tree from data is called decision tree learning.

- Internal node denotes a test on an attribute
- Branch represents an outcome of the test
- Leaf nodes represent class labels or class distribution

In data mining, trees have three more descriptive categories or names:

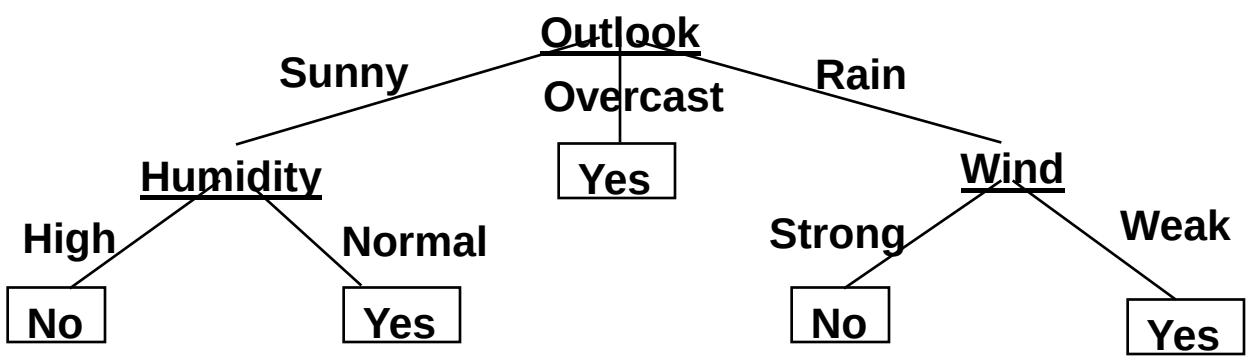
- ***Classification tree* analysis** - when the predicted outcome is the class to which the data belongs.
- ***Regression tree* analysis** - when the predicted outcome can be considered a real number (e.g. the price of a house, or a patient's length of stay in a hospital).
- ***Classification And Regression Tree* (CART) analysis** - when both of the above procedures are referred.

Decision Tree Example



Decision Tree: Example

Day	Outlook	Temperature	Humidity	Wind	Play Tennis
1	Sunny	Hot	High	Weak	No
2	Sunny	Hot	High	Strong	No
3	Overcast	Hot	High	Weak	Yes
4	Rain	Mild	High	Weak	Yes
5	Rain	Cool	Normal	Weak	Yes
6	Rain	Cool	Normal	Strong	No
7	Overcast	Cool	Normal	Strong	Yes
8	Sunny	Mild	High	Weak	No
9	Sunny	Cool	Normal	Weak	Yes
10	Rain	Mild	Normal	Weak	Yes
11	Sunny	Mild	Normal	Strong	Yes
12	Overcast	Mild	High	Strong	Yes
13	Overcast	Hot	Normal	Weak	Yes
14	Rain	Mild	High	Strong	No



Attributes = {Outlook, Temperature, Humidity, Wind}

Play Tennis = {yes, no}

Classification by Decision Tree Induction

- **Decision tree**
  - A flow-chart-like tree structure
  - Internal node denotes a test on an attribute
  - Branch represents an outcome of the test
  - Leaf nodes represent class labels or class distribution
- **Decision tree generation consists of two phases**
  - **Tree construction**
    - At start, all the training examples are at the root
    - Partition examples recursively based on selected attributes
  - **Tree pruning**
    - Identify and remove branches that reflect noise or outliers
- **Use of decision tree: Classifying an unknown sample**
  - Test the attribute values of the sample against the decision tree

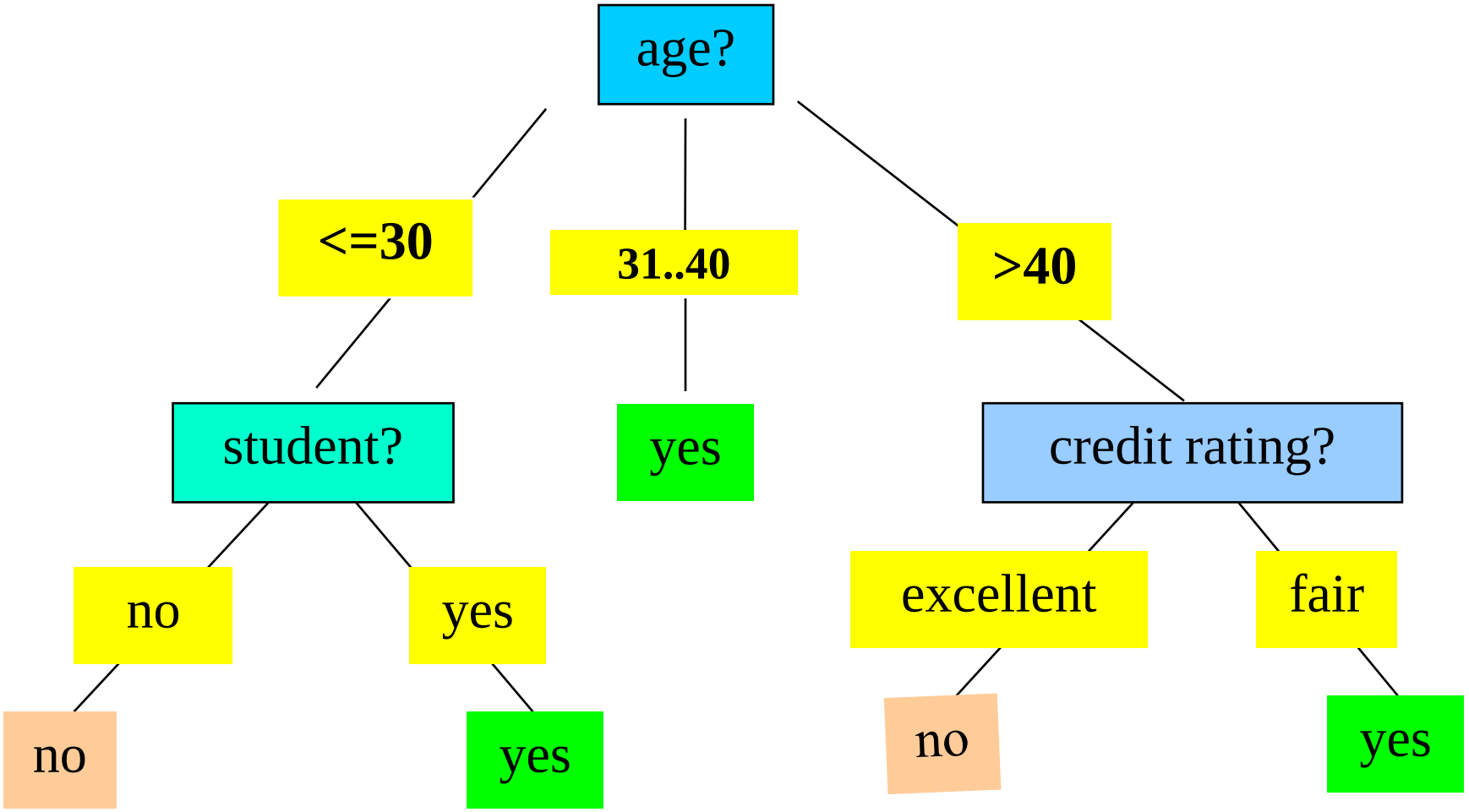
Algorithm for Decision Tree Induction

- **Basic algorithm (a greedy algorithm)**
  - Tree is constructed in a top-down recursive divide-and-conquer manner
  - At start, all the training examples are at the root
  - Attributes are categorical (if continuous-valued, they are discretized in advance)
  - Examples are partitioned recursively based on selected attributes
  - Test attributes are selected on the basis of a heuristic or statistical measure (e.g., information gain)
- **Conditions for stopping partitioning**
  - All samples for a given node belong to the same class
  - There are no remaining attributes for further partitioning – majority voting is employed for classifying the leaf
  - There are no samples left

Decision Tree Induction: Training Dataset

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

Output: A Decision Tree for "*buys_computer*"



Algorithm for building Decision Trees

Decision trees are a popular structure for supervised learning. They are constructed using attributes best able to differentiate the concepts to be learned.

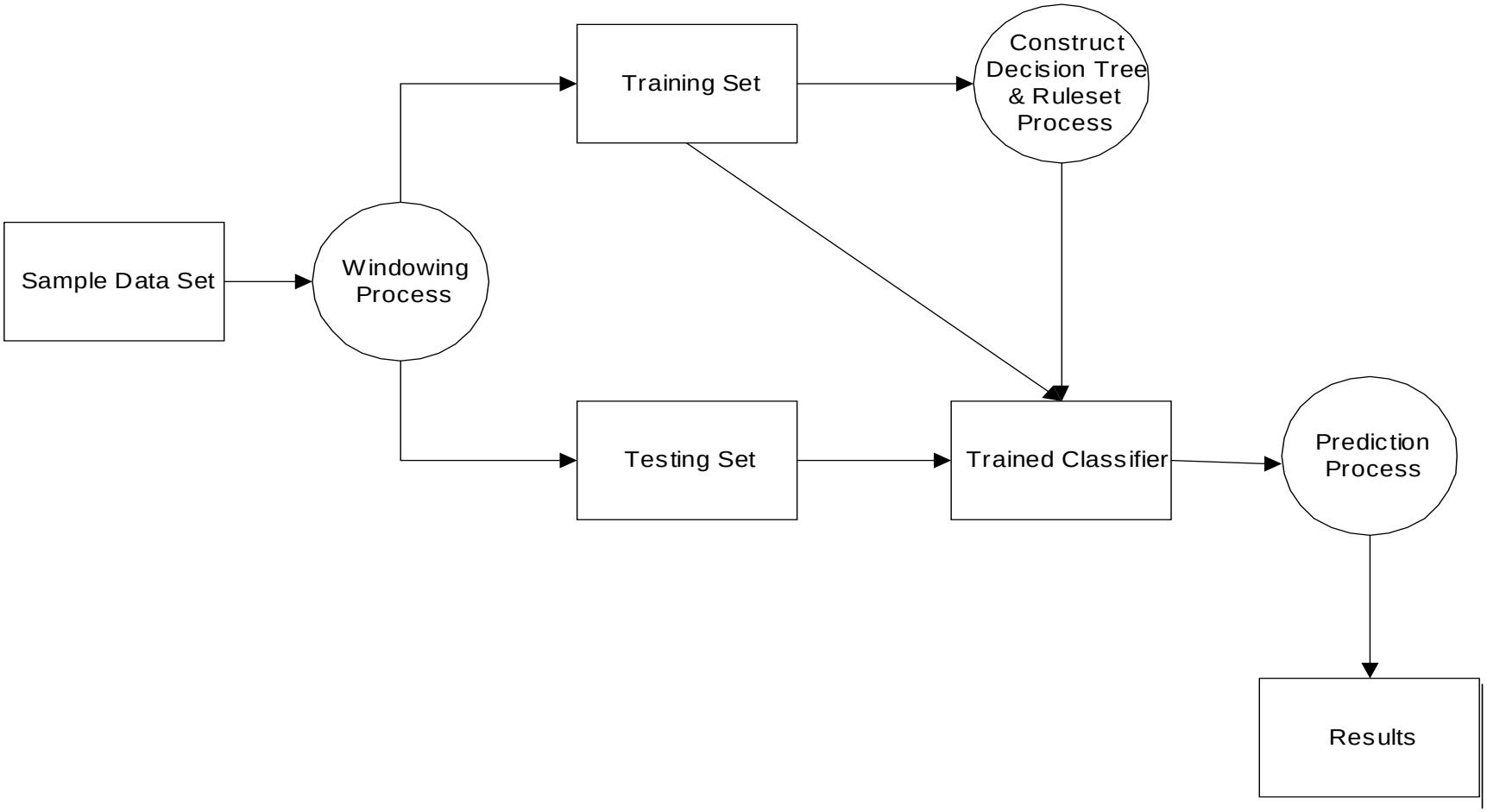
A decision tree is built by initially selecting a subset of instances from a training set. This subset is then used by the algorithm to construct a decision tree. The remaining training set instances test the accuracy of the constructed tree.

If the decision tree classified the instances correctly, the procedure terminates.

If an instance is incorrectly classified, the instance is added to the selected subset of training instances and a new tree is constructed.

This process continues until a tree that correctly classify all non-selected instances is created or the decision tree is built from the entire training set.

|-------- Data Preparation Stage --------|------- Tree Building Stage -------|--- Prediction Stage ---|



Block Diagram of Training Process

Decision Tree Algorithm

```
Input:
   T      //Decision Tree
   D      //Input Database
Output:
   M      //Model Prediction
DTProc Algorithm:
                //Illustrates Prediction Technique using DT
   for each t ∈ D do
      n = root node of T;
      while n not leaf node do
         Obtain answer to question on n applied t;
         Identify arc from t which contains correct answer;
         n = node at end of this arc;
      Make prediction for t based on labeling of n;
```

Decision Tree Pseudocode

```
node = tree-design(Data = {X,C})
        For i = 1 to d
            quality_variable(i) = quality_score(X_i, C)
        end
        node = {X_split, Threshold } for max{quality_variable}
        {Data_right, Data_left} = split(Data, X_split, Threshold)
        if node == leaf?
                return(node)
        else
                node_right = tree-design(Data_right)
                node_left = tree-design(Data_left)
        end
end
```

Basic algorithm for inducing a decision tree

Algorithm: Generate_decision_tree. Generate a decision tree from the given training data.

Input: The training samples, represented by discrete-valued attributes; the set of candidate attributes, attribute-list;

Output: A decision tree

Begin

Partition (S)

If (all records in S are of the same class or only 1 record found in S) then return;

For each attribute Ai do

evaluate splits on attribute Ai;

Use best split found to partition S into S1 and S2 to grow a tree with two Partition (S1) and Partition (S2);

Repeat partitioning for Partition (S1) and (S2) until it meets tree stop growing criteria;

End;

Decision Tree Algorithms & their main Issues

1. **Tree Structure -** Selection of a tree structure like Balanced tree for improving performance.
2. **Training Data -** Structure of a tree depends on the training data. Selecting adequate data prevents either the tree to over fit and on the other hand good enough to work on a general data
3. **Stopping Criteria -** Construction of a tree stops on a Stopping criteria. It is essential to achieve a balance between too early or late to create a tree with right level.
4. **Pruning -** After constructing a tree, modify it to remove duplication or sub-trees.
5. **Splitting -** Selection of the best splitting attribute and size of the training set are important factors in creating a decision tree algorithm. For example, Splitting attributes in the case of students may be gender, marks scored and electives chosen. The order in which splitting attributes are chosen are important for avoiding redundancy and unnecessary comparisons at different levels.

Decision Tree Learning Algorithm - ID3

ID3 (Iterative Dichotomiser)

- It is a simple decision tree learning algorithm developed by Ross Quinlan (1983).
- ID3 follow non-backtracking approach in which decision trees are constructed in a top-down recursive “divide and conquer” manner to test each attribute at every tree node.
- This approach starts with a training set of tuples and their associated class labels. Training set is recursively partitioned into smaller subsets as the tree is being built.

Algorithm for building a Decision Tree from Training Tuples

Algorithm: Generate_decision_tree. Generate a decision tree from the training tuples of data partition *D*.

Input:

- Data partition *D*, which is a set of training tuples and their associated class labels;
- *attribute_list*, the set of candidate attributes;
- *Attribute_selection method*, a procedure to determine the splitting criterion that "best" partitions the data tuples into individual classes. This criterion consists of a *splitting_attribute* and, possibly, either a *split point* or *splitting subset*.

Output: A decision tree.

ID3 Algorithm:

(1) create a node *N;*

(2) **if** tuples in *D* are all of the same class *C* **then**

(3) **return** *N* as a leaf node labeled with the class *C* ;

(4) **if** *attribute _list* is empty **then**

(5) **return** *N* as a leaf node labeled with the majority class in D; // majority voting

(6) apply **Attribute_selection_method**(*D*, *attribute_list*) to find best

(7) *splitting_criterion*; label node *N* with *splitting_criterion*;

(8) **if** *splitting attribute is discrete-valued and*

multiway splits allowed **then** // not restricted to binary trees

(9) *attribute_list* ← *attribute_list – splitting_attribute;* // remove *splitting_attribute*

(10) **for each** outcome *j* of *splitting_criterion*

// partition the tuples and grow subtrees for each partition

(11) let D_j be the set of data tuples in *D* satisfying outcome *j*; // a partition

(12) **if** D_j is empty **then**

(13) attach a leaf labeled with the majority class in *D* to node *N;*

(14) **else** attach the node returned by **Generate_decision_tree**(D_j , *attribute_list) to* node *N*;

endfor

(15) **return** *N;*

Explanation of Algorithm:

The above algorithm has three parameters D, attribute_list and attribute_selection_method. D is data partition. It is a set of training tuples and their associated class labels. Attribute_list contains a list of attributes describing the tuples.

Now tree starts as a single node N. It represents the training tuples in D. If the tuples in D are all of the same class then node N is considered as leaf. It is labeled with that class. It is occurring in step 2 and 3. Step 4 and 5 are terminating conditions. IF this condition does not follow then algorithm calls Attribute_selection_method to detemine the spilitting criterion. This criterion determines the best way to partition the tuples in D into individual classes(step 6). Step 7 serves as a test at the node. In steps 10 and 11, tuples in D are partitioned.

Advantages of using ID3

- ❖Understandable prediction rules are created from the training data.
- ❖Builds the fastest tree.
- ❖Builds a short tree.
- ❖Only need to test enough attributes until all data is classified.
- ❖Finding leaf nodes enables test data to be pruned, reducing number of tests.
- ❖Whole dataset is searched to create tree.

Disadvantages of using ID3

- Data may be over-fitted or over-classified, if a small sample is tested.
- Only one attribute at a time is tested for making a decision.
- Classifying continuous data may be computationally expensive, as many trees must be generated to see where to break the continuum.

Pros and Cons of Decision Tree

Pros

- no distributional assumptions
- can handle real and nominal inputs
- speed and scalability
- robustness to outliers and missing values
- interpretability
- compactness of classification rules
- They are easy to use.
- Generated rules are easy to understand .
- Amenable to scaling and the database size.

Cons

- several tuning parameters to set with little guidance
- decision boundary is non-continuous
- Cannot handle continuous data.
- Incapable of handling many problems which cannot be divided into attribute domains.
- Can lead to over-fitting as the trees are constructed from training data.

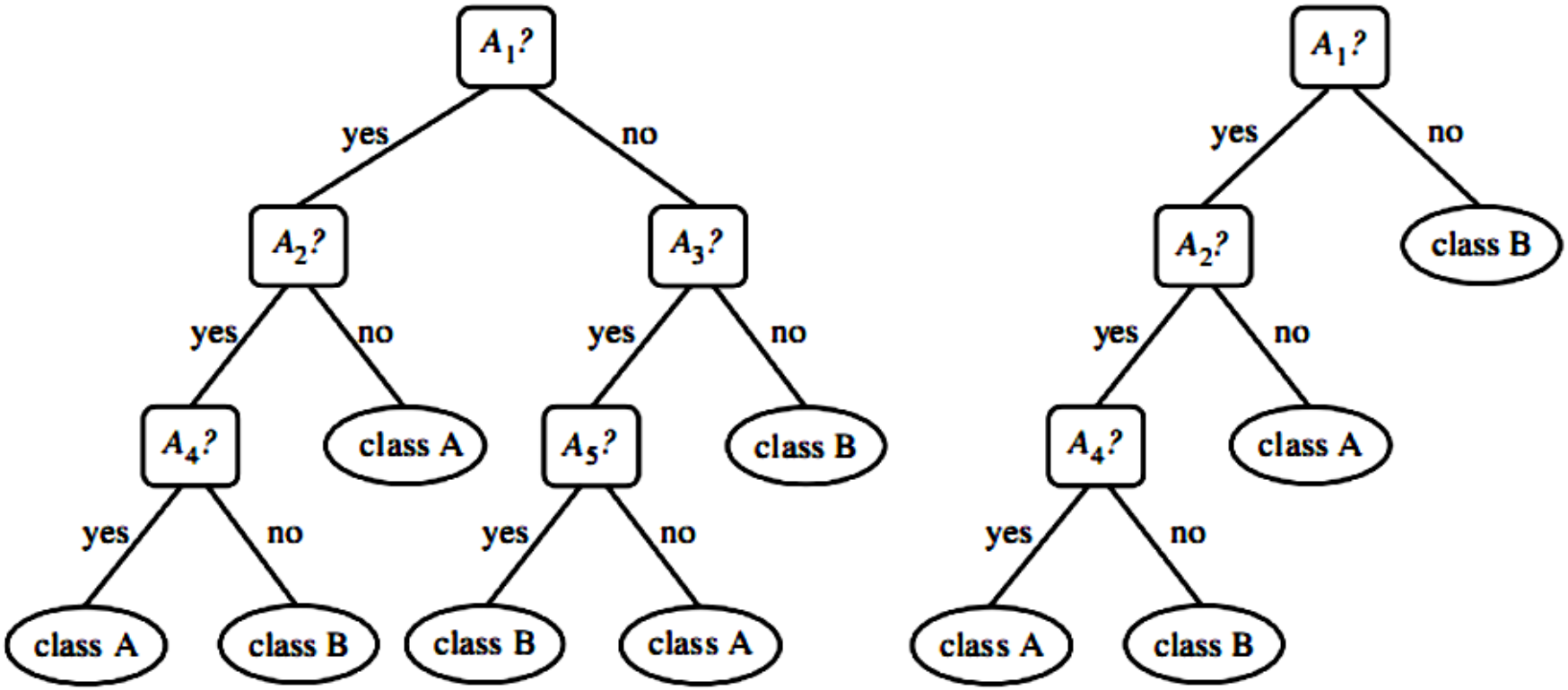
Tree Pruning

- When a decision tree is built, many of the branches will reflect anomalies in the training data due to noise or outliers. Tree pruning methods address this problem of over fitting the data.
- There are **two common approaches** for tree pruning
- – **Pre-pruning** (early stopping rule):
  - The tree is "pruned" by halting its construction early
  - Or stop algorithm before it becomes a fully grown tree
  - Most popular test: chi-squared test

Tree Pruning

- Post-pruning
- removes sub-trees from a “fully grown” tree: get a sequence of progressively pruned trees
- Possible strategies: error estimation, significance testing, MDL pruning
- preferred in practice

Example



An unpruned decision tree and a pruned version of it.

Rule Based Classification

- has learned model as a set of IF-THEN rules
- We need to
  - How to generate the model
  - Examine how to use model to classify data
- IF-THEN rule is an expression of the form
  - IF condition THEN conclusion

Ex:

- R1:IF age=youth and student=yes THEN buys_computer=yes

Or

- R1: (age=youth)Λ(student=yes)=>(buys_computer=yes)

Using If-Then rules for classification

- IF part is called rule antecedent or precondition, it can consist of one or more attributes test
- THEN part is called rule consequent, it consist a class prediction
- A rule R can be assessed by its coverage and accuracy
  - Given a tuple X from a data D
  - Let n_{cover}: # of tuples covered by R
  - $n_{correct}$: # of tuples correctly classify by R
  - |D|: # of tuples in D

$$coverage(R) = \frac{n_{covers}}{|D|}$$

$$accuracy(R) = \frac{n_{correct}}{n_{covers}}.$$

Using IF-THEN rules for classification

Ex: of assessing R

Table 6.1 Class-labeled training tuples from the *AllElectronics* customer database.

RID	*age*	*income*	*student*	*credit_rating*	*Class: buys_computer*
1	youth	high	no	fair	no
2	youth	high	no	excellent	no
3	middle_aged	high	no	fair	yes
4	senior	medium	no	fair	yes
5	senior	low	yes	fair	yes
6	senior	low	yes	excellent	no
7	middle_aged	low	yes	excellent	yes
8	youth	medium	no	fair	no
9	youth	low	yes	fair	yes
10	senior	medium	yes	fair	yes
11	youth	medium	yes	excellent	yes
12	middle_aged	medium	no	excellent	yes
13	middle_aged	high	yes	fair	yes
14	senior	medium	no	excellent	no

Using IF-THEN rules for classification

R: IF age=youth AND student=yes THEN buys_computer=yes

➔ |D| = 14

$n_{cover} = 2$

$n_{correct} = 2$

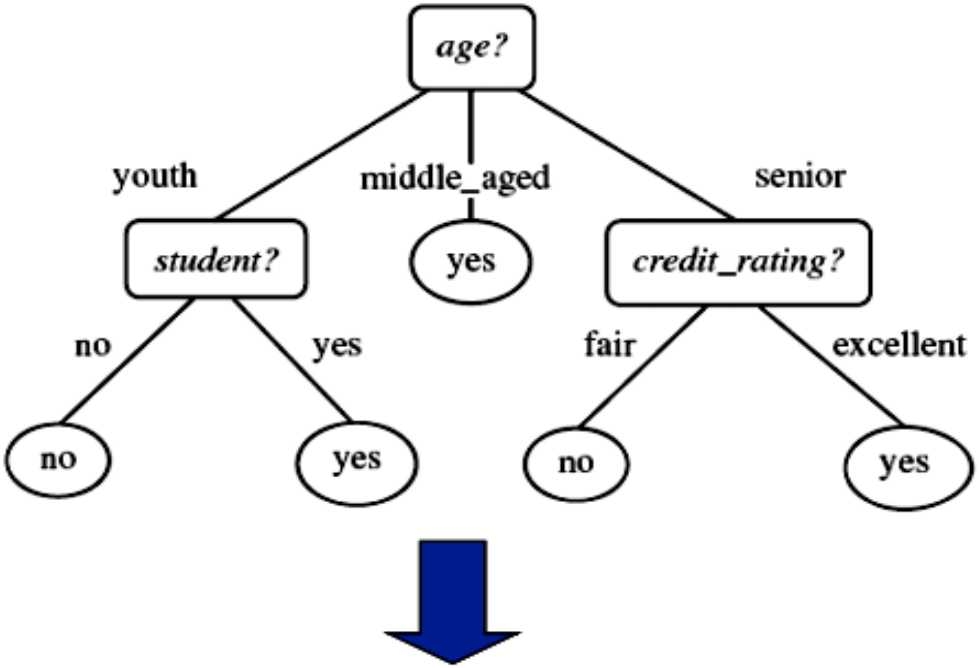
$$\text{cov}\,erage(R) = \frac{2}{14} = 14.28\%$$

$$accuracy(R) = \frac{2}{2} = 100\%$$

Rule Extraction from a Decision Tree

- One rule is created for each path from the root to a leaf node
- Each splitting criterion is logically AND to form the rule antecedent (IF part)
- Leaf node holds the class prediction for rule consequent (THEN part)
- Logical OR is implied between each of the extracted rules

Example



*R*1: IF *age* = *youth* AND *student* = *no* THEN *buys_computer* = *no*
*R*2: IF *age* = *youth* AND *student* = *yes* THEN *buys_computer* = *yes*
*R*3: IF *age* = *middle_aged* THEN *buys_computer* = *yes*
*R*4: IF *age* = *senior* AND *credit_rating* = *excellent* THEN *buys_computer* = *yes*
*R*5: IF *age* = *senior* AND *credit_rating* = *fair* THEN *buys_computer* = *no*

Bayesian Classification: Why?

- **A statistical classifier:** performs probabilistic prediction, i.e., predicts class membership probabilities
- **Foundation:** Based on Bayes' Theorem.
- **Performance:** A simple Bayesian classifier, naïve Bayesian classifier, has comparable performance with decision tree and selected neural network classifiers
- **Incremental**: Each training example can incrementally increase/decrease the probability that a hypothesis is correct- prior knowledge can be combined with observed data
- **Standard:** Even when Bayesian methods are computationally intractable, they can provide a standard of optimal decision making against which other methods can be measured

Bayesian Theorem: Basics

- Let **X** be a data sample ("evidence"): class label is unknown
- Let H be a hypothesis that **X** belongs to class C
- Classification is to determine P(H|**X**) (posteriori probability of H conditioned on **X**), the probability that the hypothesis holds given the observed data sample **X**
- P(H) (prior probability), the initial probability
  - E.g., H is "will buy computer" regardless of age, income, …
- P(**X**): prior probability that an evidence is observed
  - E.g., **X** is "age in 31..40, medium income"
- P(**X**|H) (posteriori probability of **X** conditioned on H), the probability of observing the evidence **X**, given that the hypothesis holds
  - E.g., Given H that a customer will buy computer, P(**X**|H) is the prob. of evidence **X** that customer's age is 31..40, medium income

Bayesian Theorem

- Given training evidence **X**, *posteriori probability of a hypothesis* H, P(H|**X**), follows the Bayes theorem

$$P(H|\mathbf{X}) = \frac{P(\mathbf{X}|H)P(H)}{P(\mathbf{X})}$$

- Informally, this can be written as

  posteriori = likelihood x prior/evidence

- Predicts **X** belongs to C_i iff the probability $P(C_i|\mathbf{X})$ is the highest among all the $P(C_k|X)$ for all the *k* classes
- Practical difficulty: require initial knowledge of many probabilities, significant computational cost

Towards Naïve Bayesian Classifier

- Let D be a training set of tuples and their associated class labels, and each tuple is represented by an n-dim. attribute vector **X** = (x_1, x_2, …, x_n)
- Suppose there are *m* classes C_1, C_2, …, C_m.
- Classification is to derive the maximum posteriori, i.e., the maximal $P(C_i|\mathbf{X})$
- This can be derived from Bayes' theorem

$$P(C_i|\mathbf{X}) = \frac{P(\mathbf{X}|C_i)P(C_i)}{P(\mathbf{X})}$$

- Since prior prob. P(**X**) is constant for all classes,

maximizing $P(C_i|\mathbf{X})$ is equivalent to maximizing $P(\mathbf{X}|C_i)P(C_i)$

Naïve Bayesian Classifier: Training Dataset

Class:
C_1:buys_computer = 'yes'
C_2:buys_computer = 'no'

Data sample
X = (age = youth,
income = medium,
student = yes
credit_rating = Fair)

age	income	student	credit_rating	comp
youth	high	no	fair	no
youth	high	no	excellent	no
middle	high	no	fair	yes
senior	medium	no	fair	yes
senior	low	yes	fair	yes
senior	low	yes	excellent	no
middle	low	yes	excellent	yes
youth	medium	no	fair	no
youth	low	yes	fair	yes
senior	medium	yes	fair	yes
youth	medium	yes	excellent	yes
middle	medium	no	excellent	yes
middle	high	yes	fair	yes
senior	medium	no	excellent	no

Naïve Bayesian Classifier: An Example

- $P(C_i)$: P(buys_computer = "yes") = 9/14 = 0.643
  P(buys_computer = "no") = 5/14= 0.357

- Compute $P(X|C_i)$ for each class
  P(age = "youth" | buys_computer = "yes") = 2/9 = 0.222
  P(age = "youth" | buys_computer = "no") = 3/5 = 0.6
  P(income = "medium" | buys_computer = "yes") = 4/9 = 0.444
  P(income = "medium" | buys_computer = "no") = 2/5 = 0.4
  P(student = "yes" | buys_computer = "yes) = 6/9 = 0.667
  P(student = "yes" | buys_computer = "no") = 1/5 = 0.2
  P(credit_rating = "fair" | buys_computer = "yes") = 6/9 = 0.667
  P(credit_rating = "fair" | buys_computer = "no") = 2/5 = 0.4

- **X = (age = youth, income = medium, student = yes, credit_rating = fair)**

$P(X|C_i)$: P(X|buys_computer = "yes") = 0.222 x 0.444 x 0.667 x 0.667 = 0.044
P(X|buys_computer = "no") = 0.6 x 0.4 x 0.2 x 0.4 = 0.019
$P(X|C_i)*P(C_i)$: P(X|buys_computer = "yes") * P(buys_computer = "yes") = 0.028
P(X|buys_computer = "no") * P(buys_computer = "no") = 0.007

Therefore, **X** belongs to class ("buys_computer = yes")

What Is Prediction?

- (Numerical) prediction is similar to classification
  - construct a model
  - use model to predict continuous or ordered value for a given input
- Prediction is different from classification
  - Classification refers to predict categorical class label
  - Prediction models continuous-valued functions
- Major method for prediction: regression
  - model the relationship between one or more *independent* or **predictor** variables and a *dependent* or **response** variable
- Regression analysis
  - Linear and multiple regression
  - Non-linear regression
  - Other regression methods: generalized linear model, Poisson regression, log-linear models, regression trees

Linear Regression

- Linear regression: involves a response variable y and a single predictor variable x

  $y = w_0 + w_1 x$

  where w_0 (y-intercept) and w_1 (slope) are regression coefficients
- Method of least squares: estimates the best-fitting straight line

$$w_1 = \frac{\sum_{i=1}^{|D|}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{|D|}(x_i - \bar{x})^2} \qquad w_0 = \bar{y} - w_1 \bar{x}$$

- Multiple linear regression: involves more than one predictor variable
  - Training data is of the form ($\mathbf{X_1}$, y_1), ($\mathbf{X_2}$, y_2),…, ($\mathbf{X_{|D|}}$, $y_{|D|}$)
  - Ex. For 2-D data, we may have: $y = w_0 + w_1 x_1 + w_2 x_2$
  - Solvable by extension of least square method or using SAS, S-Plus
  - Many nonlinear functions can be transformed into the above

Linear Regression

A regression model is comprised of a dependent, or response, variable and an independent, or predictor, variable.

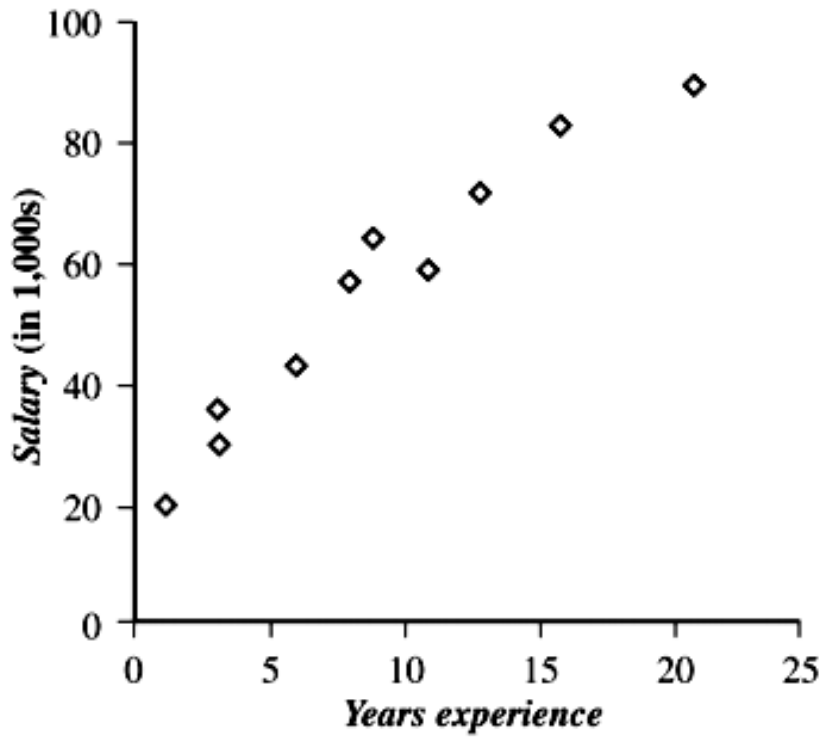
Dependent Variable = Independent Variable(s)

Prediction Relationship

Linear Regression

Salary data.

x years experience	y salary (in \$1000s)
3	30
8	57
9	64
13	72
3	36
6	43
11	59
21	90
1	20
16	83



Plot of data: Although the points do not fall on a straight line, the overall pattern suggests a linear relationship between x(years experience) and y (salary)

Linear Regression

Given the above data, we compute $\bar{x} = 9.1$ and $\bar{y} = 55.4$. Substituting these values into Equations (6.50) and (6.51), we get

$$w_1 = \frac{(3-9.1)(30-55.4)+(8-9.1)(57-55.4)+\cdots+(16-9.1)(83-55.4)}{(3-9.1)^2+(8-9.1)^2+\cdots+(16-9.1)^2} = 3.5$$

$$w_0 = 55.4-(3.5)(9.1) = 23.6$$

Thus, the equation of the least squares line is estimated by $y = 23.6 + 3.5x$. Using this equation, we can predict that the salary of a college graduate with, say, 10 years of experience is \$58,600. ∎

Nonlinear Regression

- Some nonlinear models can be modeled by a polynomial function
- A polynomial regression model can be transformed into linear regression model. For example,

$$y = w_0 + w_1 x + w_2 x^2 + w_3 x^3$$

convertible to linear with new variables: $x_2 = x^2$, $x_3 = x^3$

$$y = w_0 + w_1 x + w_2 x_2 + w_3 x_3$$

- Other functions, such as power function, can also be transformed to linear model
- Some models are intractable nonlinear (e.g., sum of exponential terms)
  - possible to obtain least square estimates through extensive calculation on more complex formulae

Other Regression-Based Models

- **Generalized linear model:**
  - Foundation on which linear regression can be applied to modeling categorical response variables
  - Variance of y is a function of the mean value of y, not a constant
  - Logistic regression: models the prob. of some event occurring as a linear function of a set of predictor variables
  - Poisson regression: models the data that exhibit a Poisson distribution
- **Log-linear models:** (for categorical data)
  - Approximate discrete multidimensional prob. distributions
  - Also useful for data compression and smoothing
- **Regression trees and model trees**
  - Trees to predict continuous values rather than class labels

Regression Trees and Model Trees

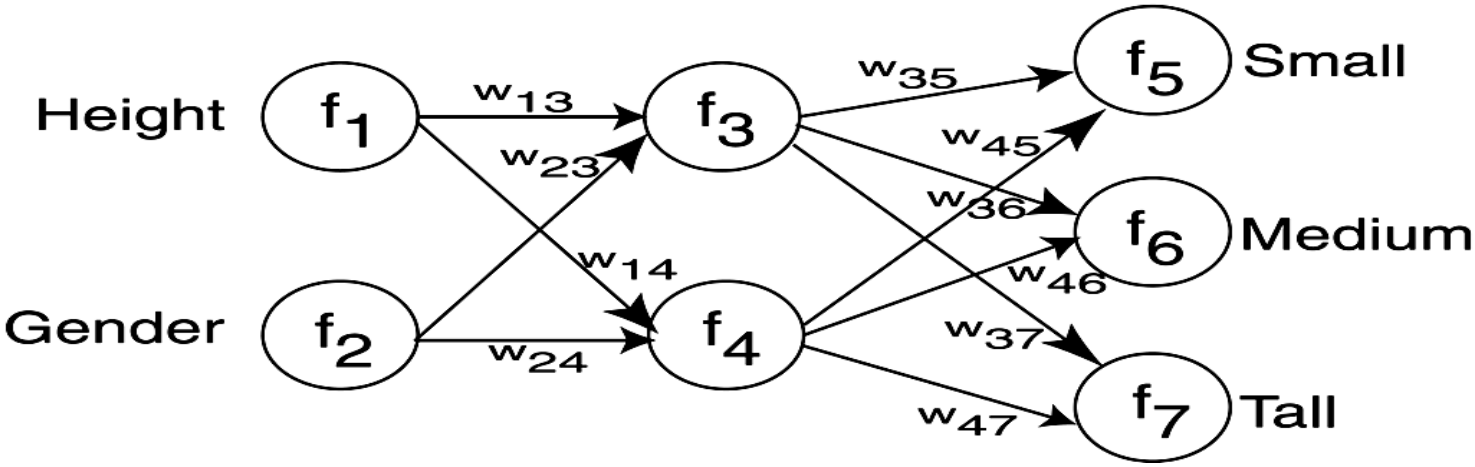
- **Regression tree: proposed in CART system (Breiman et al. 1984)**
  - CART: Classification And Regression Trees
  - Each leaf stores a *continuous-valued prediction*
  - It is the *average value of the predicted attribute* for the training tuples that reach the leaf
- **Model tree: proposed by Quinlan (1992)**
  - Each leaf holds a regression model—a multivariate linear equation for the predicted attribute
  - A more general case than regression tree
- **Regression and model trees** tend to be more accurate than linear regression when the data are not represented well by a simple linear model

Neural Networks

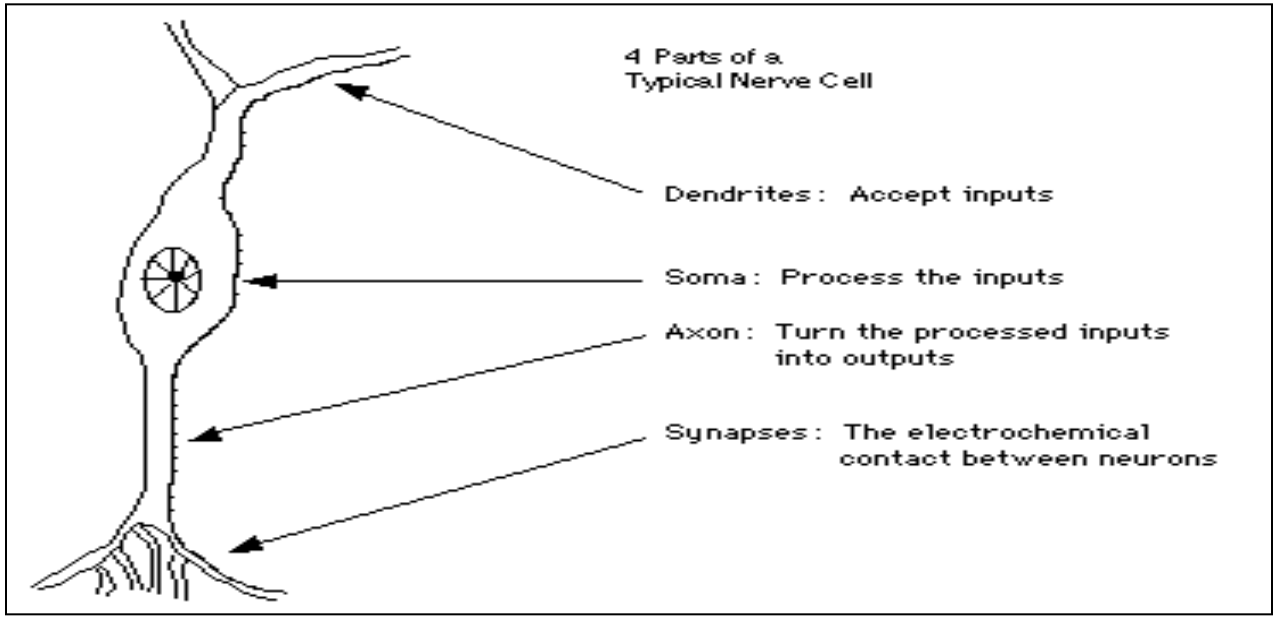
- Neural Network is a set of connected INPUT/OUTPUT UNITS, where each connection has a WEIGHT associated with it.
- It is a case of SUPERVISED, INDUCTIVE or CLASSIFICATION learning.
- Neural Network learns by adjusting the weights so as to be able to correctly classify the training data and hence, after testing phase, to classify unknown data.
- Neural Network needs long time for training.
- Neural Network has a high tolerance to noisy and incomplete data.

A Neural Network (NN) is a directed graph F=<V,A> with vertices V={1,2,…,n} and arcs A={<i,j>|1<=i,j<=n}, with the following restrictions:

- V is partitioned into a set of input nodes, V_I, hidden nodes, V_H, and output nodes, V_O.
- The vertices are also partitioned into layers
- Any arc <i,j> must have node i in layer h-1 and node j in layer h.
- Arc <i,j> is labeled with a numeric value w_{ij}.
- Node i is labeled with a function f_i.

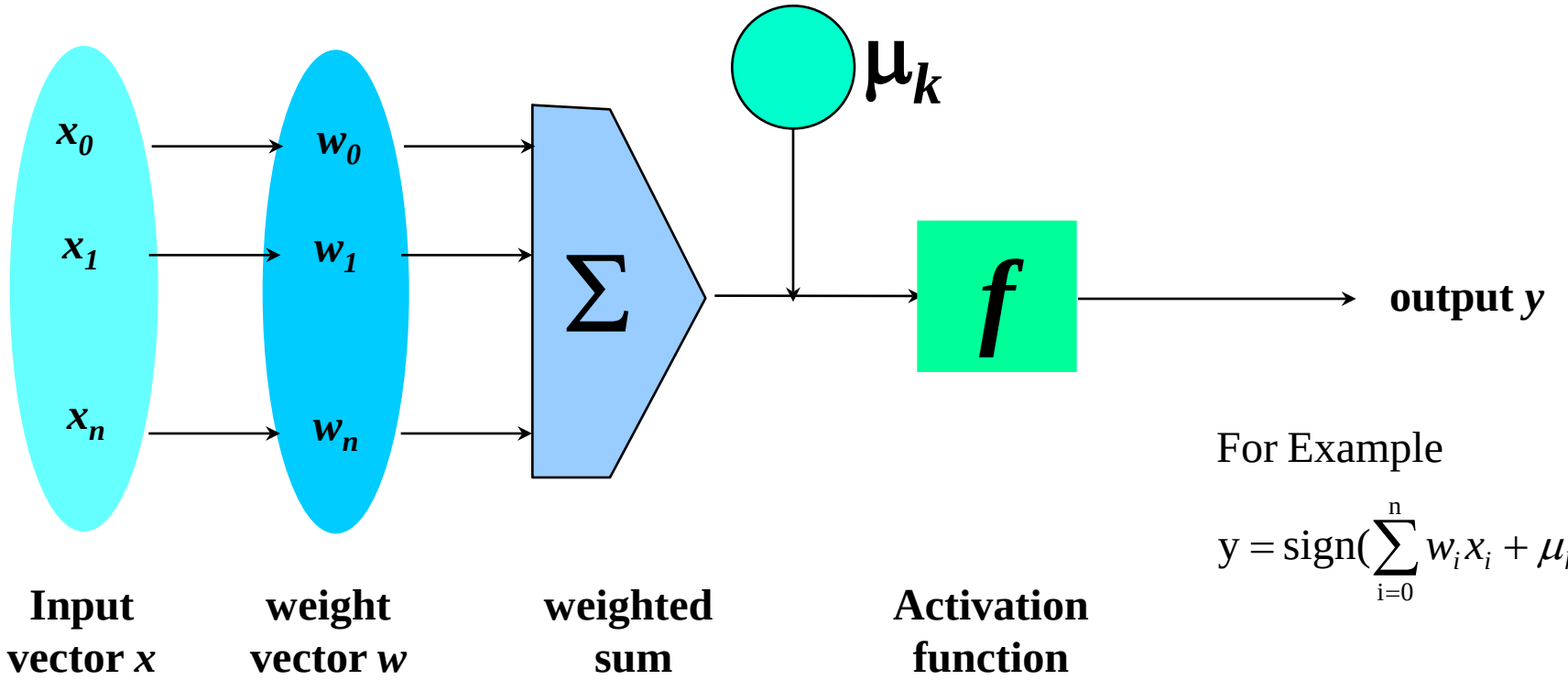


Similarity with Biological Network



- Fundamental processing element of a neural network is a neuron
- A human brain has 100 billion neurons
- An ant brain has 250,000 neurons

A Neuron (= a Perceptron)



For Example

$$y = \text{sign}\left(\sum_{i=0}^{n} w_i x_i + \mu_k\right)$$

The *n*-dimensional input vector *x* is mapped into variable *y* by means of the scalar product and a nonlinear function mapping

Network Training

The ultimate objective of training is to obtain a set of weights that makes almost all the tuples in the training data classified correctly.

Steps

Initialize weights with random values

Feed the input tuples into the network one by one

For each unit

- Compute the net input to the unit as a linear combination of all the inputs to the unit
- Compute the output value using the activation function
- Compute the error
- Update the weights and the bias

Backpropagation

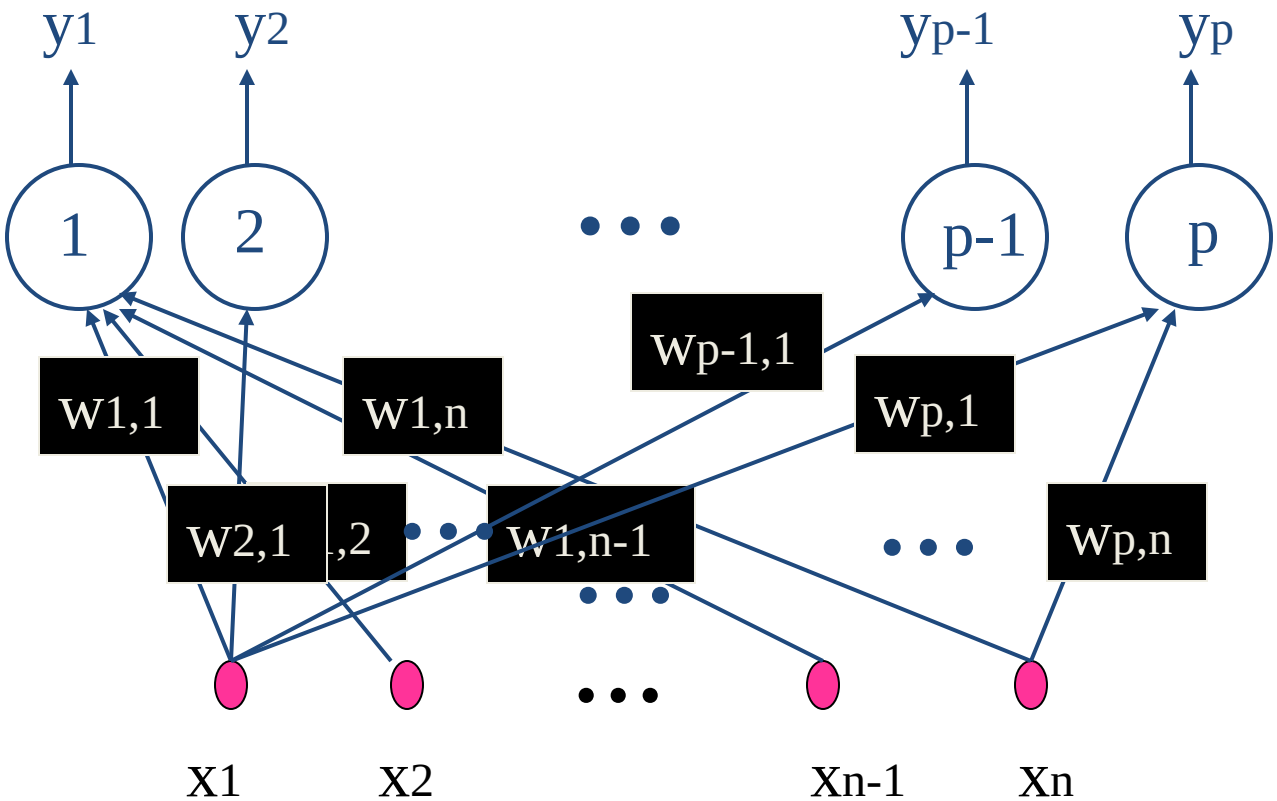
- Backpropagation is a neural network learning algorithm which gained reputed in the 1980s.
- Backpropagation learns by iteratively processing a data set of training tuples, comparing the network's prediction for each tuple with the actual known *target* value.
- The target value may be the known class label of the training tuple (for classification problems) or a continuous value (for prediction).
- For each training tuple, the weights are modified so as to minimize the mean squared error between the network's prediction and the actual target value.

- These modifications are made in the “backwards” direction, that is, from the output layer, through each hidden layer down to the first hidden layer (hence the name *backpropagation*).
- Although it is not guaranteed, in general the weights will eventually converge, and the learning process stops.

Steps in Backpropagation:

- Initialize weights (to small random numbers) and biases in the network
- Propagate the inputs forward (by applying activation function)
- Backpropagate the error (by updating weights and biases)
- Terminating condition (when error is very small, etc.)

Perceptron



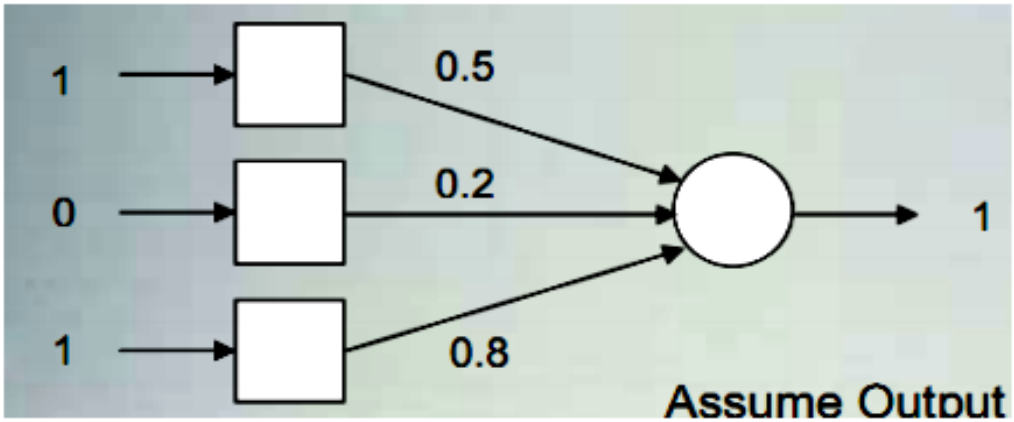
$$y_i(t+1) = f\left(\sum_{k=0}^{n} w_{ik}\, x_k(t)\right) \qquad i = 1, 2, \ldots p$$

How Do Perceptrons Learn?

Uses supervised training

- **If the output is not correct, the weights are adjusted according to the formula:**

$w_{new} = w_{old} + \alpha(\text{desired} - \text{output})*\text{input}$ α is the learning rate



$1 * 0.5 + 0 * 0.2 + 1 * 0.8 = 1.3$

Assuming Output Threshold = 1.2

$1.3 > 1.2$

Assume $\alpha = 1$

W1new = 0.5 + 1*(0-1)*1 = -0.5

W2new = 0.2 + 1*(0-1)*0 = 0.2

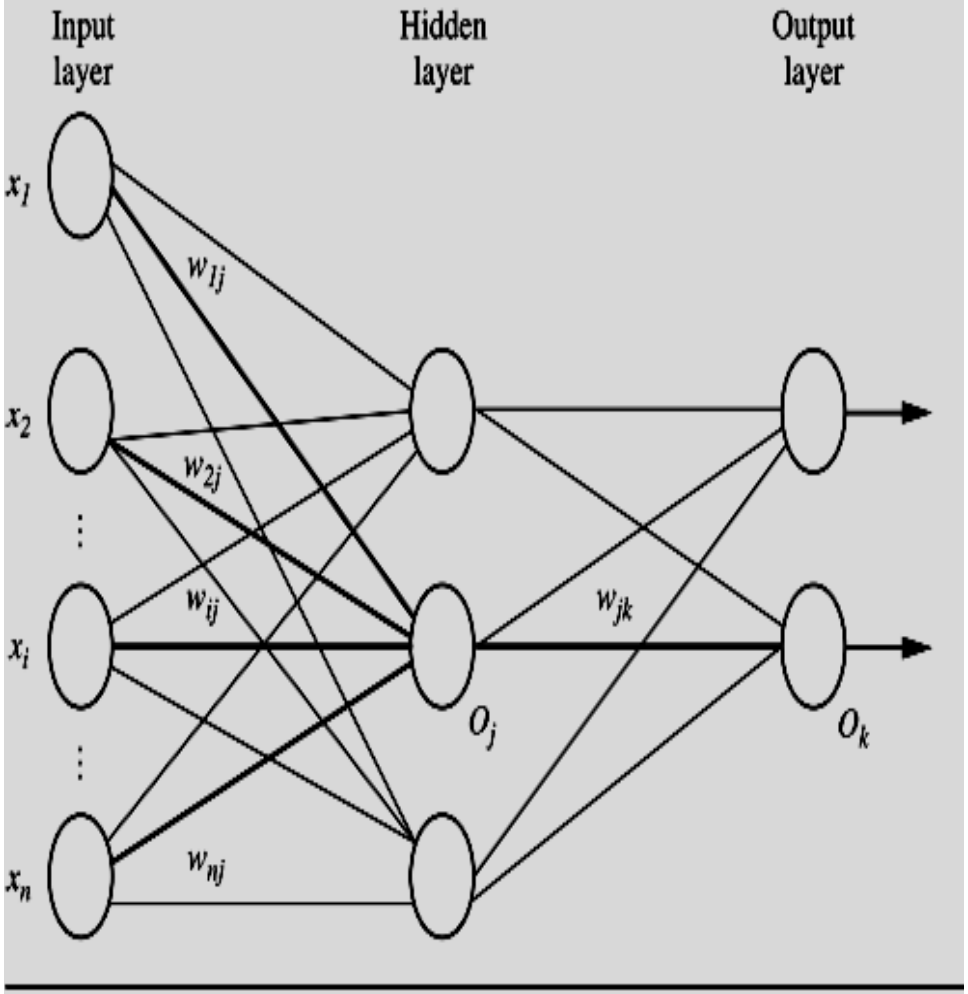
W = 0.8 + 1*(0-1)*1 = -0.2

Multilayer Feed Forward Networks

- Most common neural network
  - An extension of the perceptron
    - ► Multiple layers
- The addition of one or more “hidden” layers in between the input and output layers
  - ► Activation function is not simply a threshold
- Usually a sigmoid function
  - ► A general function approximate
- Not limited to linear problems
  - Information flows in one direction
    - ► The outputs of one layer act as inputs to the next layer

Multilayer Feed-Forward NN

- The Backpropagation algorithm performs learning on a multilayer feed-forward neural network. It iteratively learns a set of weights for prediction of the class label of tuples.
- A multilayer feed-forward neural network consists of an input layer, one or more hidden layers, and an output layer.
- An example of a multilayer feed-forward network is shown in Figure



A multilayer feed-forward neural network.

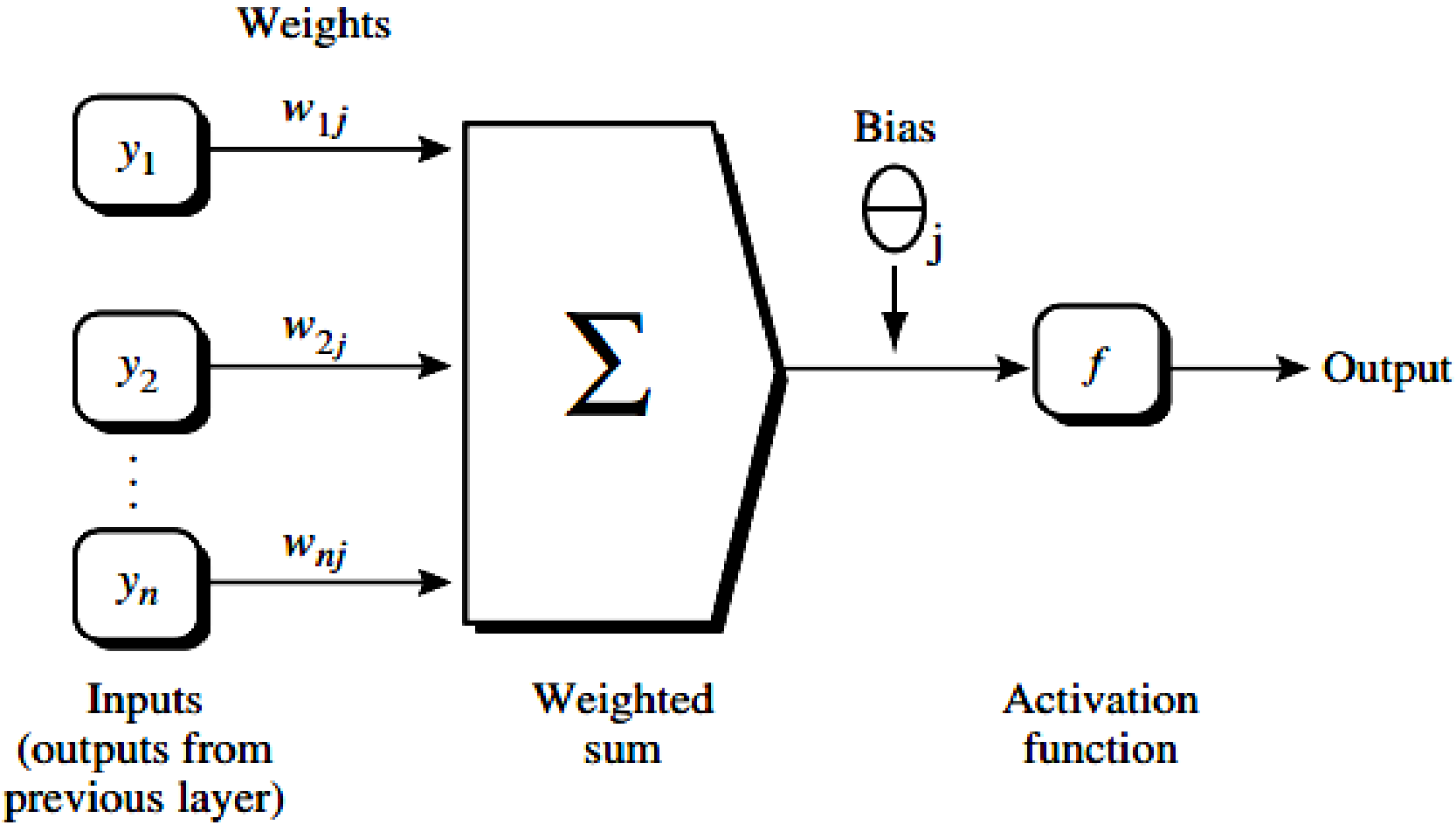
Multilayer Feed-Forward NN cont..

- Each layer is made up of units. The inputs to the network correspond to the attributes measured for each training tuple.
- The inputs are fed simultaneously into the units making up the input layer. These inputs pass through the input layer and are then weighted and fed simultaneously to a second layer of “neuron like” units, known as a hidden layer.
- The outputs of the hidden layer units can be input to another hidden layer, and so on. The number of hidden layers is arbitrary, although in practice, usually only one is used.
- The weighted outputs of the last hidden layer are input to units making up the output layer, which emits the network’s prediction for given tuples.
- The units in the input layer are called input units. The units in the hidden layers and output layer are sometimes referred to as neurodes, due to their symbolic biological basis, or as output units.
- The multilayer neural network shown in Figure above has two layers of output units.
- Therefore, we say that it is a two-layer neural network. (The input layer is not counted because it serves only to pass the input values to the next layer.)

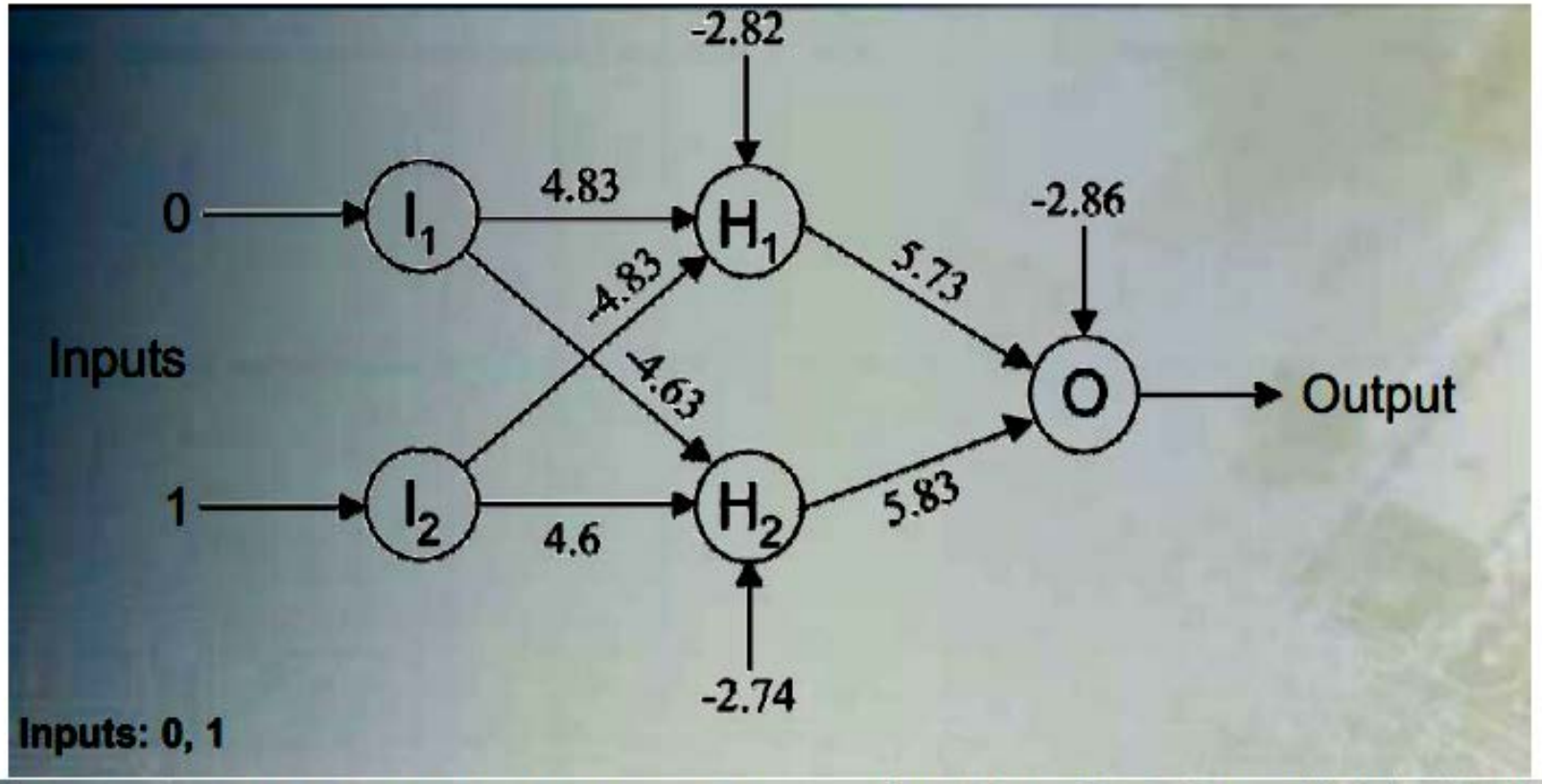
Multilayer Feed-Forward NN cont..

- Similarly, a network containing two hidden layers is called a three-layer neural network, and so on.
- The network is feed-forward in that none of the weights cycles back to an input unit or to an output unit of a previous layer. It is fully connected in that each unit provides input to each unit in the next forward layer.
- Each output unit takes, as input, a weighted sum of the outputs from units in the previous layer (shown in Figure in next slide). It applies a nonlinear (activation) function to the weighted input.
- Multilayer feed-forward neural networks are able to model the class prediction as a nonlinear combination of the inputs.
- From a statistical point of view, they perform nonlinear regression.
- Multilayer feed-forward networks, given enough hidden units and enough training samples, can closely approximate any function.

Multilayer Feed-Forward NN cont..



XOR Example



H_1: Net = 0(4.83) + 1(-4.83) – 2.82 = -7.65
Output = $1 / (1 + e^{7.65}) = 4.758 \times 10^{-4}$

H_2: Net = 0(-4.63) + 1(4.6) – 2.74 = 1.86
Output = $1 / (1 + e^{-1.86}) = 0.8652$

O: Net = $4.758 \times 10^{-4}(5.73) + 0.8652(5.83) - 2.86 = 2.187$
Output = $1 / (1 + e^{-2.187}) = 0.8991 =$ "1"

Advantages of Neural Network

- prediction accuracy is generally high
- robust, works when training examples contain errors
- output may be discrete, real-valued, or a vector of several discrete or real-valued attributes
- fast evaluation of the learned target function
- High tolerance to noisy data
- Ability to classify untrained patterns
- Well-suited for continuous-valued inputs and outputs
- Successful on a wide array of real-world data
- Algorithms are inherently parallel
- Techniques have recently been developed for the extraction of rules from trained neural networks

Disadvantages of Neural Network

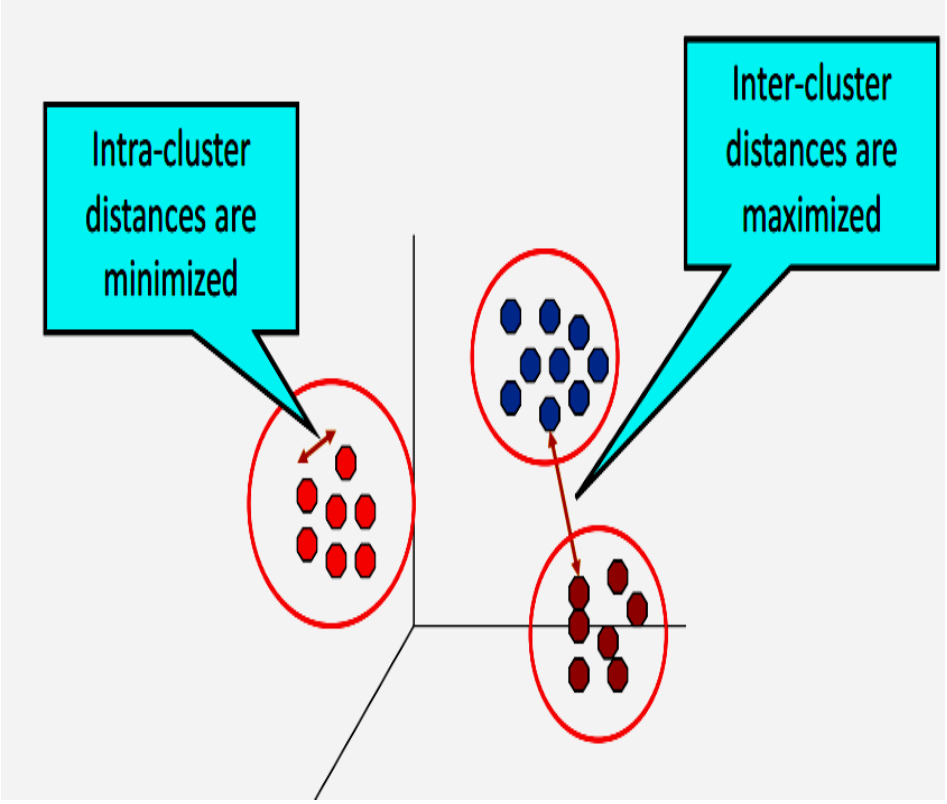
- long training time
- difficult to understand the learned function (weights)
- not easy to incorporate domain knowledge
- Require a number of parameters typically best determined empirically, e.g., the network topology or ``structure."
- *Poor interpretability:* Difficult to interpret the symbolic meaning behind the learned weights and of ``hidden units" in the network

What is Cluster Analysis?

- **Cluster:** a collection of data objects
  - Similar to one another within the same cluster
  - Dissimilar to the objects in other clusters
- **Cluster analysis**
  - Grouping a set of data objects into clusters
- Clustering is unsupervised classification: no predefined classes
- Clustering is used:
  - As a stand-alone tool to get insight into data distribution
    - Visualization of clusters may unveil important information
  - As a preprocessing step for other algorithms
    - Efficient indexing or compression often relies on clustering

What is Cluster Analysis?

- Finding groups of objects such that the objects in a group will be similar (or related) to one another and different from (or unrelated to) the objects in other groups



General Applications of Clustering

- **Pattern Recognition**
- **Spatial Data Analysis**
  - create thematic maps in GIS by clustering feature spaces
  - detect spatial clusters and explain them in spatial data mining
- **Image Processing**
  - cluster images based on their visual content
- **Economic Science (especially market research)**
- **WWW and IR**
  - document classification
  - cluster Weblog data to discover groups of similar access patterns

Examples of Clustering Applications

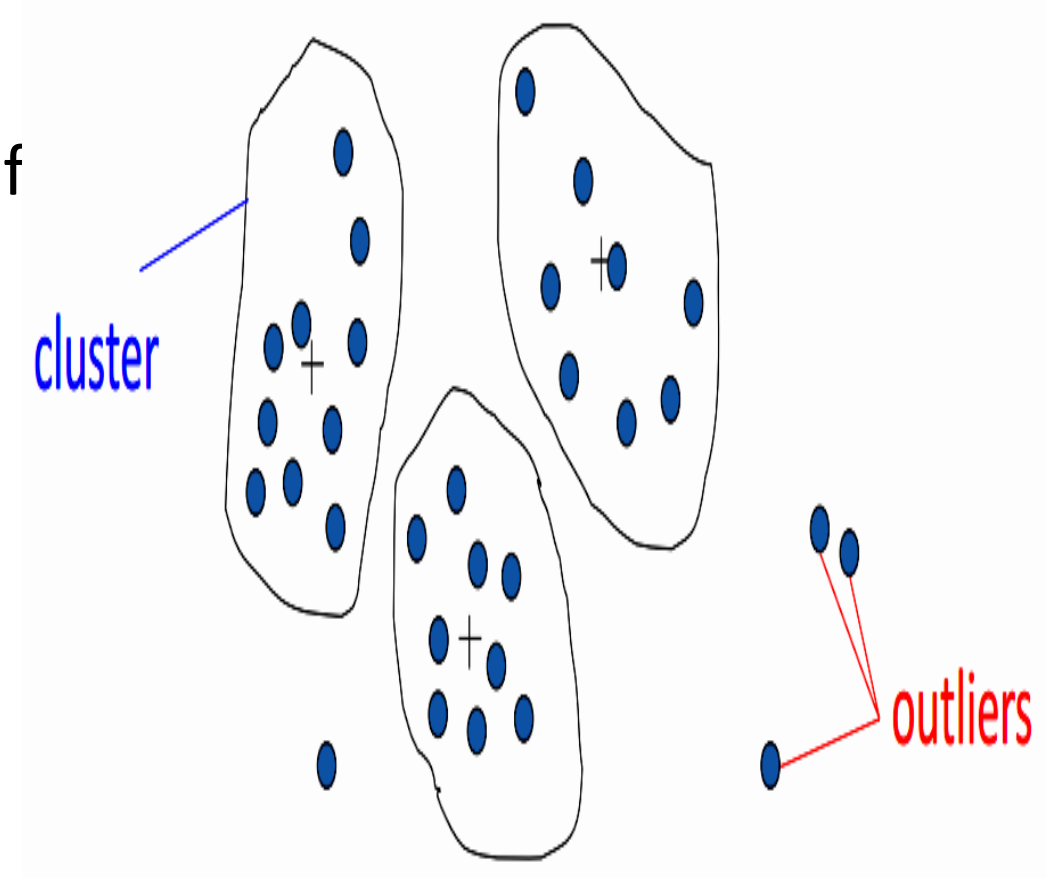
- **Marketing:** Help marketers discover distinct groups in their customer bases, and then use this knowledge to develop targeted marketing programs
- **Land use:** Identification of areas of similar land use in an earth observation database
- **Insurance:** Identifying groups of motor insurance policy holders with a high average claim cost
- **City-planning:** Identifying groups of houses according to their house type, value, and geographical location

What Is Good Clustering?

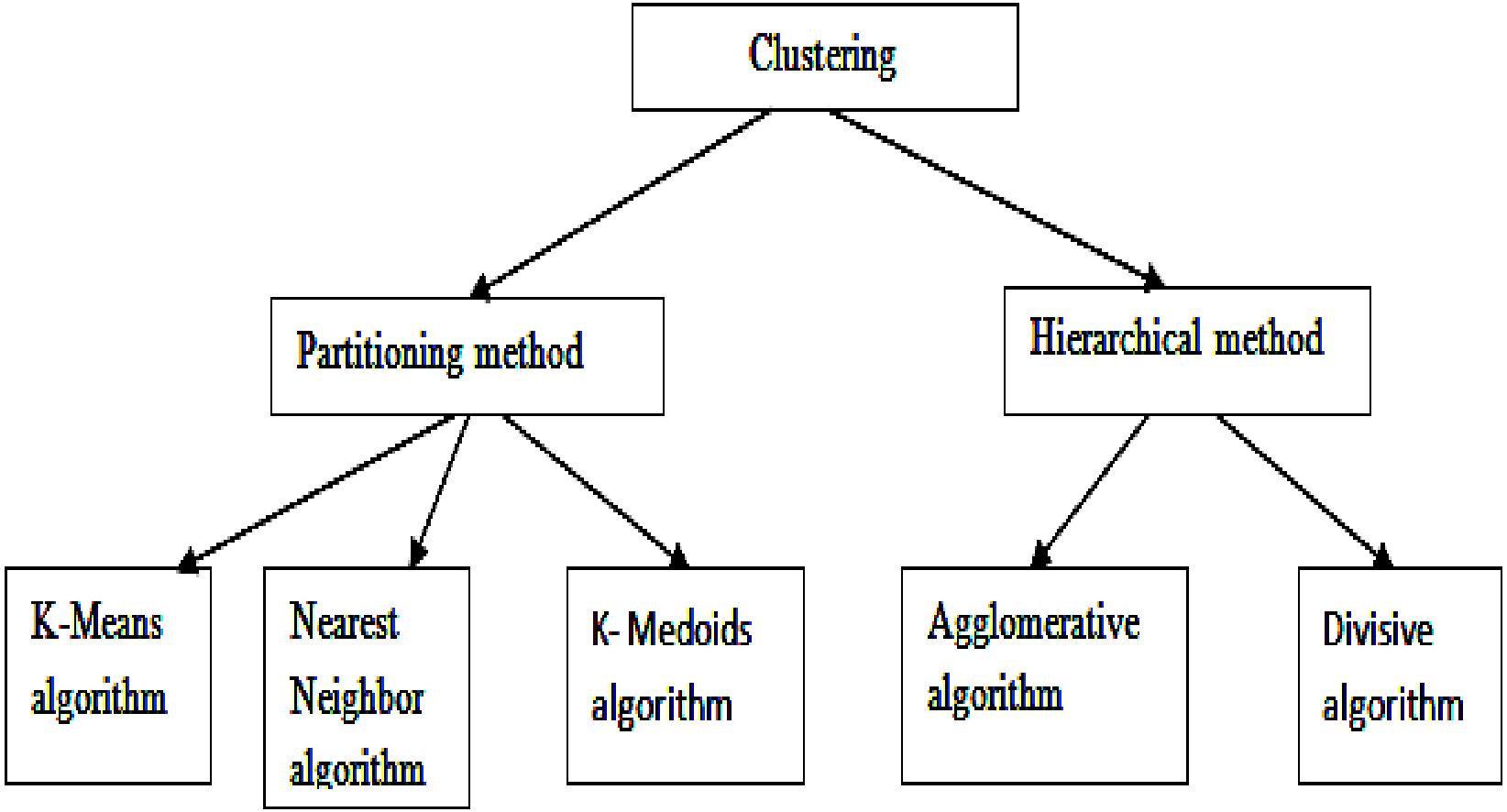
- **A good clustering** method will produce high quality clusters with
  - high intra-class similarity
  - low inter-class similarity
- The quality of a clustering result depends on both the **similarity measure** used by the method and its implementation.
- The quality of a clustering method is also measured by its ability to discover some or all of the **hidden patterns**.

Outliers

- Outliers are objects that do not belong to any cluster or form clusters of very small cardinality
- In some applications we are interested in discovering outliers, not clusters (outlier analysis)



Types of Clustering



Types of Clustering

1. **Partitioning method:**

- In this method, a partitioning algorithm arranges all the objects into various partitions, where total number of partitions is less then the total number of objects for examples, a database of n objects can be arranged into k partitions where k<n. Here each partition represents cluster. There are 3-types of partitioning algorithm:-
  - K-means algorithm
  - K-Medoids algorithm
  - Nearest Neighbor algorithm

2. **Hierarchical method**

- A hierarchical clustering method works by grouping data objects into a tree of clusters. It is classified into two types, they are:
  - Agglomerative hierarchical clustering
  - Divisive hierarchical clustering

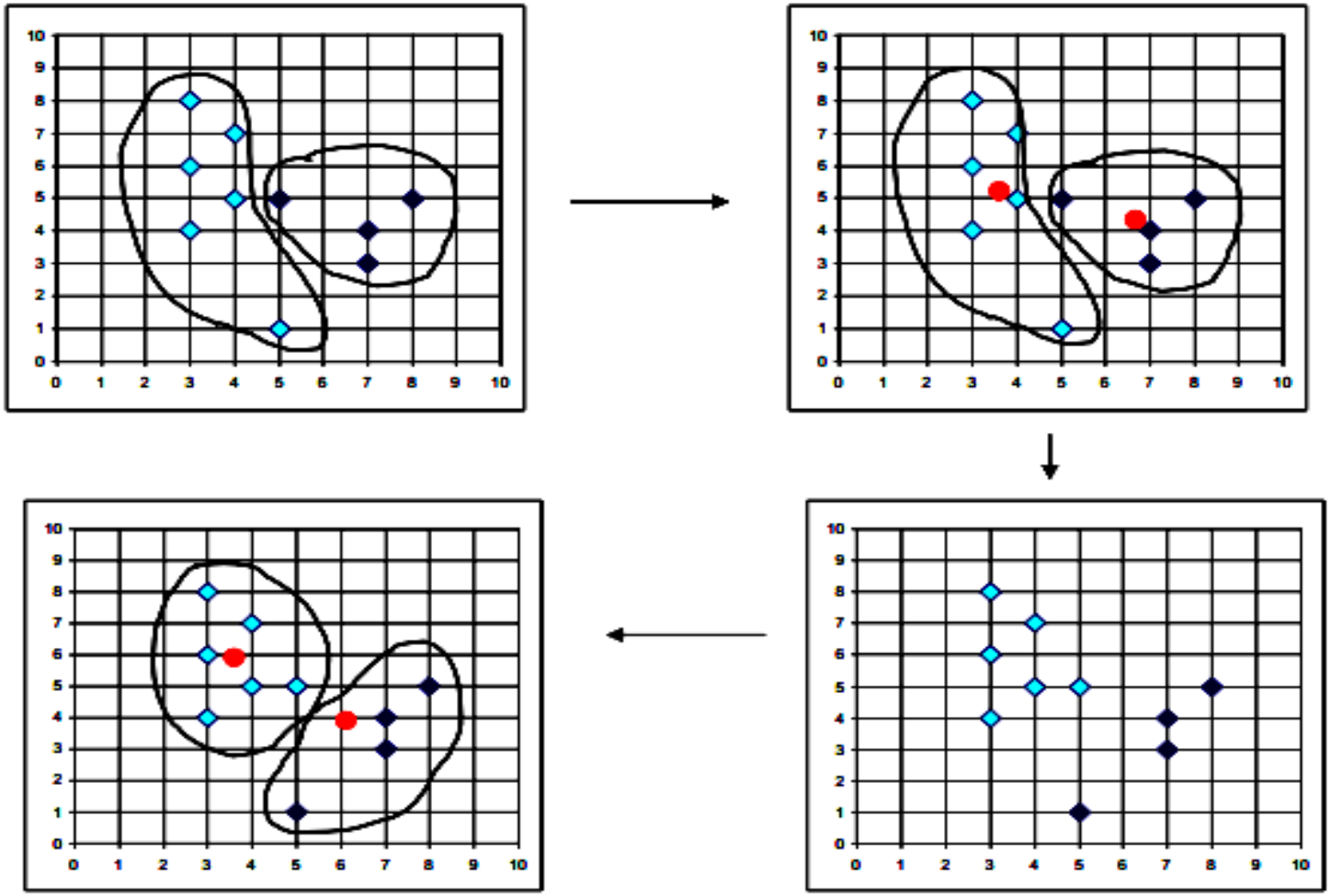
The k-means Clustering Method

- **Given k, the k-means algorithm is implemented in 4 steps:**

1. Partition objects into k nonempty subsets
2. Compute seed points as the centroids of the clusters of the current partition. The **centroid** is the center (mean point) of the cluster.
3. Assign each object to the cluster with the nearest seed point.
4. Go back to Step 2, stop when no more new assignment.

The k-means Clustering Method

Example



K-Means example

- **2, 3, 6, 8, 9, 12, 15, 18, 22 – break into 3 clusters**
  - Cluster 1 - 2, 8, 15 – mean = 8.3
  - Cluster 2 - 3, 9, 18 – mean = 10
  - Cluster 3 - 6, 12, 22 – mean = 13.3
- **Re-assign**
  - Cluster 1 - 2, 3, 6, 8, 9 – mean = 5.6
  - Cluster 2 – mean = 0
  - Cluster 3 – 12, 15, 18, 22 – mean = 16.75
- **Re-assign**
  - Cluster 1 – 3, 6, 8, 9 – mean = 6.5
  - Cluster 2 – 2 – mean = 2
  - Cluster 3 = 12, 15, 18, 22 – mean = 16.75

K-Means example (continued)

- **Re-assign**
  - Cluster 1 - 6, 8, 9 – mean = 7.6
  - Cluster 2 – 2, 3 – mean = 2.5
  - Cluster 3 – 12, 15, 18, 22 – mean = 16.75
- **Re-assign**
  - Cluster 1 - 6, 8, 9 – mean = 7.6
  - Cluster 2 – 2, 3 - mean = 2.5
  - Cluster 3 – 12, 15, 18, 22 – mean = 16.75
- **No change, so we're done**

K-Means example different starting order

- **2, 3, 6, 8, 9, 12, 15, 18, 22 – break into 3 clusters**
  - Cluster 1 - 2, 12, 18 – mean = 10.6
  - Cluster 2 - 6, 9, 22 – mean = 12.3
  - Cluster 3 – 3, 8, 15 – mean = 8.6
- **Re-assign**
  - Cluster 1 - mean = 0
  - Cluster 2 – 12, 15, 18, 22 - mean = 16.75
  - Cluster 3 – 2, 3, 6, 8, 9 – mean = 5.6
- **Re-assign**
  - Cluster 1 – 2 – mean = 2
  - Cluster 2 – 12, 15, 18, 22 – mean = 16.75
  - Cluster 3 = 3, 6, 8, 9 – mean = 6.5

K-Means example (continued)

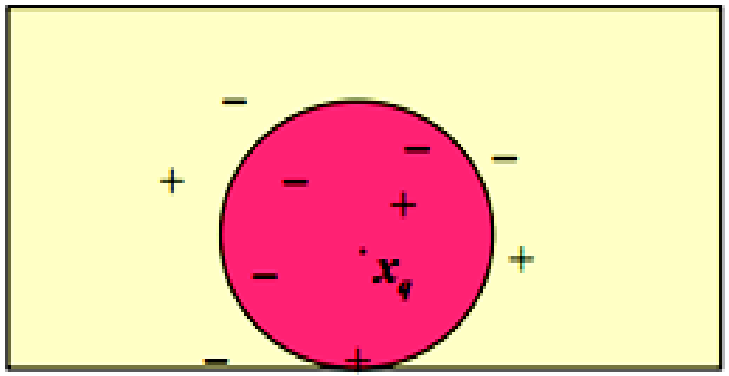
- **Re-assign**
  - Cluster 1 – 2, 3 – mean = 2.5
  - Cluster 2 – 12, 15, 18, 22 – mean = 16.75
  - Cluster 3 – 6, 8, 9 – mean = 7.6
- **Re-assign**
  - Cluster 1 – 2, 3 – mean = 2.5
  - Cluster 2 – 12, 15, 18, 22 - mean = 16.75
  - Cluster 3 – 6, 8, 9 – mean = 7.6
- **No change, so we're done**

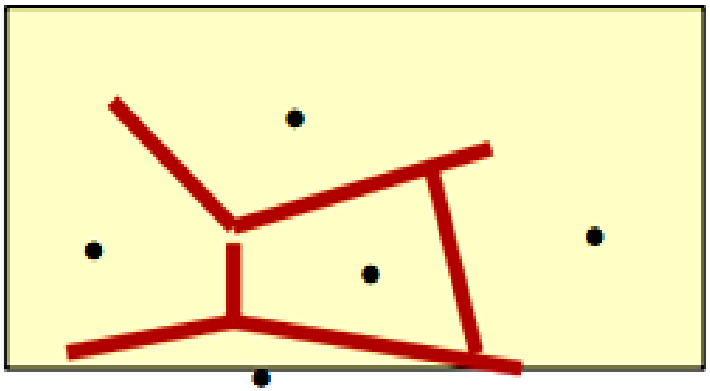
Comments on the k-means Method

- **Strength**
- Relatively efficient: O(tkn), where n is # objects, k is # clusters, and t is # iterations. Normally, k, t << n.
- **Weaknesses**
  - Applicable only when mean is defined, then what about categorical data?
  - Need to specify k, the number of clusters, in advance
  - Unable to handle noisy data and outliers

The *K*-Nearest Neighbor (K-NN) Algorithm

- All instances correspond to points in the n-D space
- The nearest neighbor are defined in terms of Euclidean distance, dist($\mathbf{X_1}$, $\mathbf{X_2}$)
- Target function could be discrete- or real- valued
- For discrete-valued, *k*-NN returns the most common value (= majority vote) among the *k* training examples nearest to x_q
- Vonoroi diagram: the decision surface induced by 1-NN for a typical set of training examples





K-Nearest Neighbor

- Features
  - All instances correspond to points in an n-dimensional Euclidean space
  - Classification is delayed till a new instance arrives
  - Classification done by comparing feature vectors of the different points
  - Target function may be discrete or real-valued

Discussion on the *k*-NN Algorithm

- k-NN for real-valued prediction for a given unknown tuple
  - Returns the mean values of the *k* nearest neighbors
- Distance-weighted nearest neighbor algorithm
  - Weight the contribution of each of the k neighbors according to their distance to the query x_q
    - Give greater weight to closer neighbors

$$w \equiv \frac{1}{d(x_q, x_i)^2}$$

- Robust to noisy data by averaging k-nearest neighbors
- Curse of dimensionality: distance between neighbors could be dominated by irrelevant attributes
  - To overcome it, axes stretch or elimination of the least relevant attributes
- Categorical (non-numerical) attributes, e.g., color, are difficult to handle

Example

X1= Acid Durability (seconds)	X2= Strength (kg/square meter)	Y= Classification
7	7	Bad
7	4	Bad
3	4	Good
1	4	Good

Now the factory produces a new paper tissue that pass laboratory test with X1 = 3 and X2 = 7. Without another expensive survey, can we guess what the classification of this new tissue is?

Contd..

1. Determine parameter K = number of nearest neighbors, assume k=3
2. Calculate the distance between the query-instance and all the training samples

Coordinate of query instance is (3, 7), instead of calculating the distance we compute square distance which is faster to calculate (without square root)

Contd..

X1 = Acid Durability (seconds)	X2 = Strength (kg/square meter)	Square Distance to query instance (3, 7)
7	7	$(7-3)^2+(7-7)^2=16$
7	4	$(7-3)^2+(4-7)^2=25$
3	4	$(3-3)^2+(4-7)^2=9$
1	4	$(1-3)^2+(4-7)^2=13$

Contd..

3. Sort the distance and determine nearest neighbors based on the K-th minimum distance

X1 = Acid Durability (seconds)	X2 = Strength (kg/square meter)	Square Distance to query instance (3, 7)	Rank minimum distance	Is it included in 3-Nearest neighbors?
7	7	$(7-3)^2+(7-7)^2=16$	3	Yes
7	4	$(7-3)^2+(4-7)^2=25$	4	No
3	4	$(3-3)^2+(4-7)^2=9$	1	Yes
1	4	$(1-3)^2+(4-7)^2=13$	2	Yes

Contd..

4. Gather the category Y *of the nearest neighbors.* Notice in the second row last column that the category of nearest neighbor (Y) is not included because the rank of this data is more than 3 (=K).

X1 = Acid Durability (seconds)	X2 = Strength (kg/square meter)	Square Distance to query instance (3, 7)	Rank minimum distance	Is it included in 3-Nearest neighbors?	Y = Category of nearest Neighbor
7	7	$(7-3)^2+(7-7)^2=16$	3	Yes	Bad
7	4	$(7-3)^2+(4-7)^2=25$	4	No	-
3	4	$(3-3)^2+(4-7)^2=9$	1	Yes	Good
1	4	$(1-3)^2+(4-7)^2=13$	2	Yes	Good

5. *Use simple majority of the category of nearest neighbors as the prediction value of the query instance*

We have 2 good and 1 bad, since 2>1 then we conclude that a new paper tissue that pass laboratory test with X1 = 3 and X2 = 7 is included in **Good** category.

K- Medoids Algorithm

- It is also called the partitioning around Medoids (PAM) algorithm. This algorithm represents a cluster by a Medoids instead of center cluster.
- The partitioning is based on the sum of the dissimilarities between each objects and its reference point (Medoids).

The K-Medoids algorithms works on following steps:

I. Randomly selected k- objects that represent a reference point, Medoids.

II. Assign remaining objects to a cluster that is most similar to its Medoids.

III. Randomly select a non- Medoids object orandom.

IV. Calculate the total cost C of swapping the medoid Oi with non-Medoids object orandom by cost function that is used to determine the dissimilarity between orandom and Oi.

V. Swap medoid Oi with the non-Medoids object orandom to make new Medoids, if total cost of swapping is negative.

VI. Process is again started from step 2 and it is repeated until no swapping occurs.

Agglomerative and Divisive hierarchical clustering

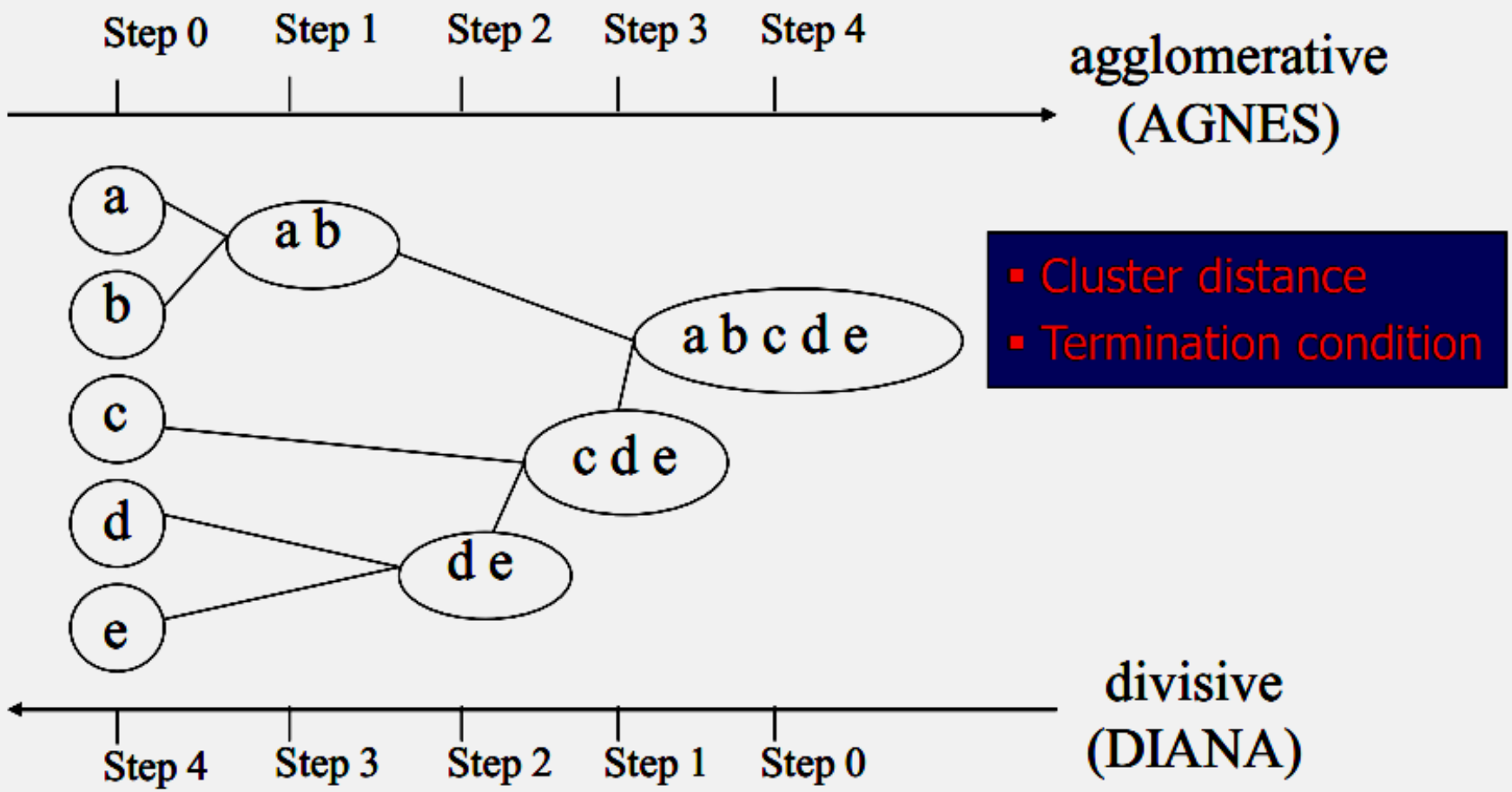
Agglomerative

- It works on the bottom-up approach. In this method, each object creates its own clusters. The single clusters are merged to make large cluster and the process of merging continues until all the singular clusters are merged into one big cluster that consists of all the objects.

Divisive

- It works on the top-down approach. In this method all the objects are arranged with in a big single cluster and the large cluster is continuously divided into smaller clusters until each cluster has a single object.

Example



- Cluster distance
- Termination condition

Thank you !!!